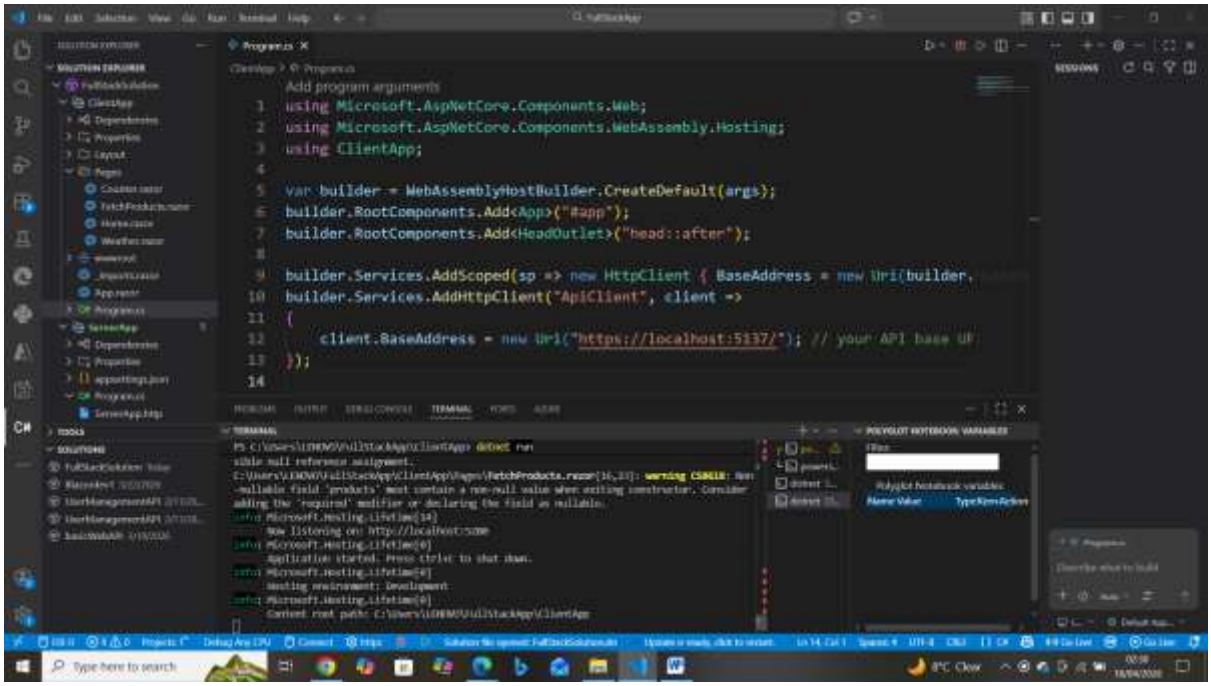
# SOLUTION TO INVENTORY HUB

# CLIENTAPP




PROGRAM.CS

```
using Microsoft.AspNetCore.Components.Web;
using Microsoft.AspNetCore.Components.WebAssembly.Hosting;
using ClientApp;
var builder = WebAssemblyHostBuilder.CreateDefault(args);
builder.RootComponents.Add<App>("#app");
builder.RootComponents.Add<HeadOutlet>("head::after");
builder.Services.AddScoped(sp => new HttpClient { BaseAddress = new
Uri(builder.HostEnvironment.BaseAddress) });
builder.Services.AddHttpClient("ApiClient", client =>
{
    client.BaseAddress = new Uri("https://localhost:5137/"); // your API base URL
});
await builder.Build().RunAsync();
```

# FetchProducts.cs

```
@page "/fetchproducts"

@inject HttpClient Http
<h3>Product List</h3>
<ul> @if (products != null)
        {
             foreach(var product in products)
                        {
                <li>@product.Name - $@product.Price</li>
            }
         }else{
          <li>Loading...</li>
           }
        </ul>
@code{
    private Product[] products;
     protected override async Task OnInitializedAsync()
      {
        // API call logic will go here
         products = await
Http.GetFromJsonAsync<Product[]>("http://localhost:5137/api/products");
      }
    public class Product
     {
         public int Id { get; set; }
         public string? Name { get; set; }
         public double Price { get; set; }
         public int Stock { get; set; }
         }
     }
```

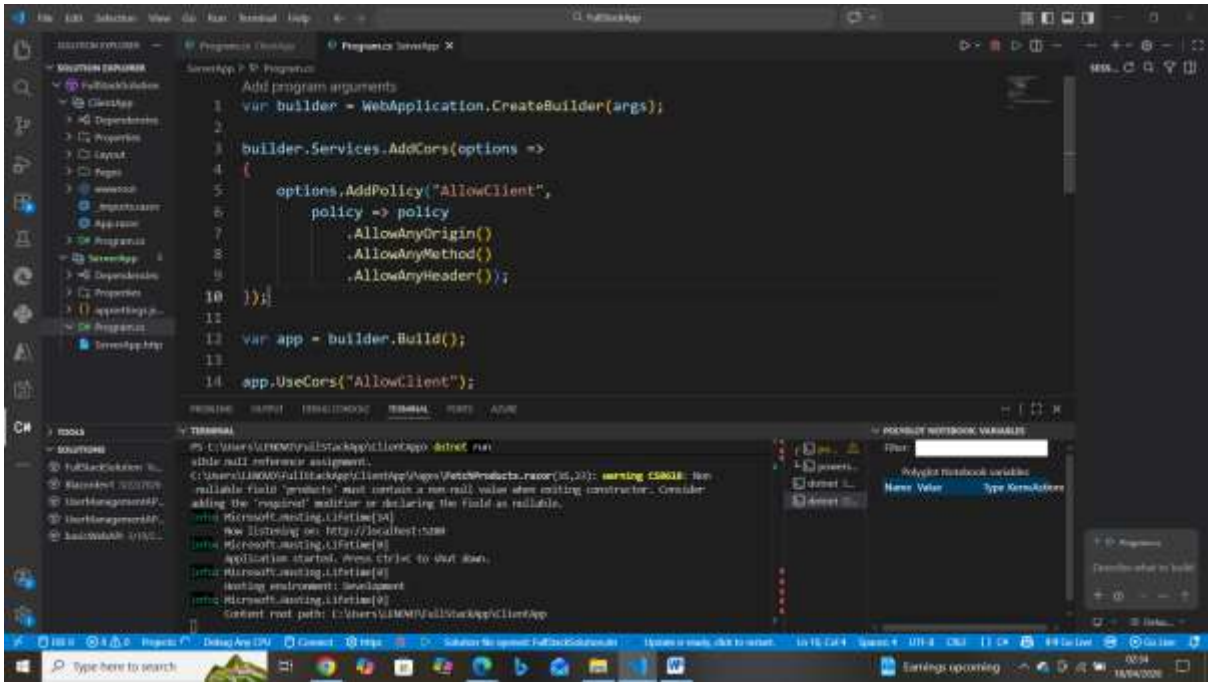
```
@page "/fetchproducts"
@inject HttpClient Http
<h3>Product List</h3>
<ul> @if (products != null)
        {
            foreach(var product in products)
                {
                <li>@product.Name - $@product.Price</li>
            }
        }else{
         <li>Loading...</li>
         }
        </ul>

@code{
```

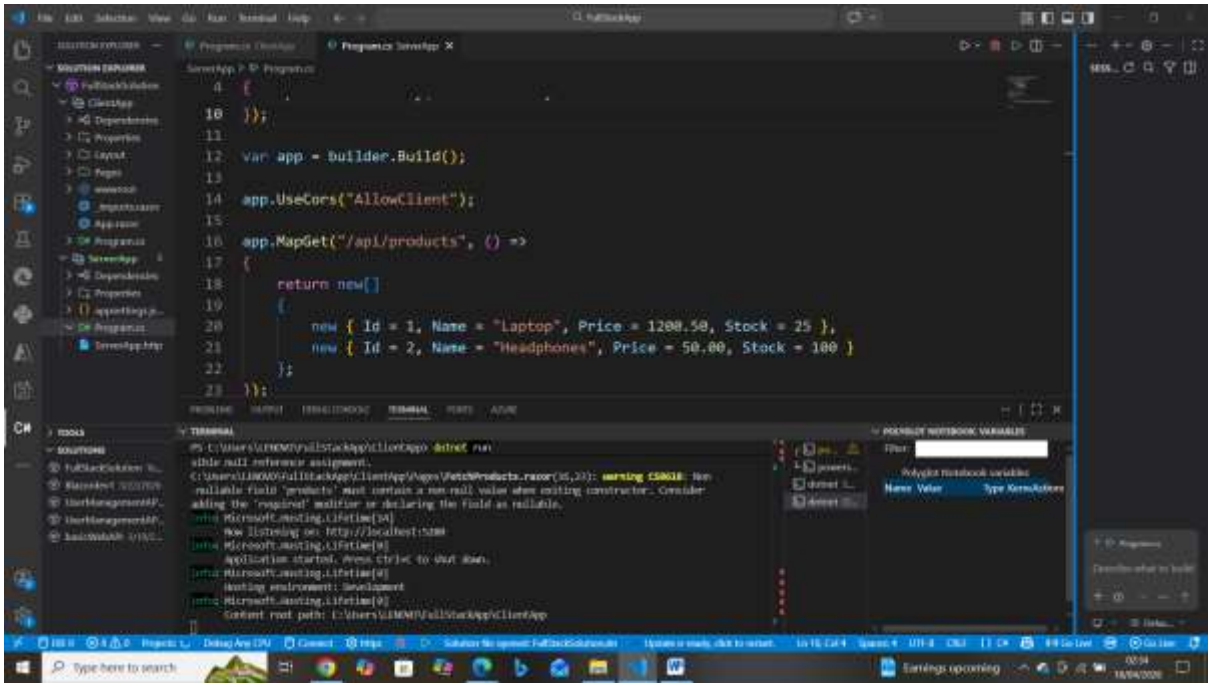
```
@code{
    private Product[] products;
     protected override async Task OnInitializedAsync()
      {
        // API call logic will go here
        products = await Http.GetFromJsonAsync<Product[]>("http://localhost:5137/api/pr
      }
    public class Product
     {
        public int Id { get; set; }
        public string? Name { get; set; }
        public double Price { get; set; }
        public int Stock { get; set; }
        }
     }
```

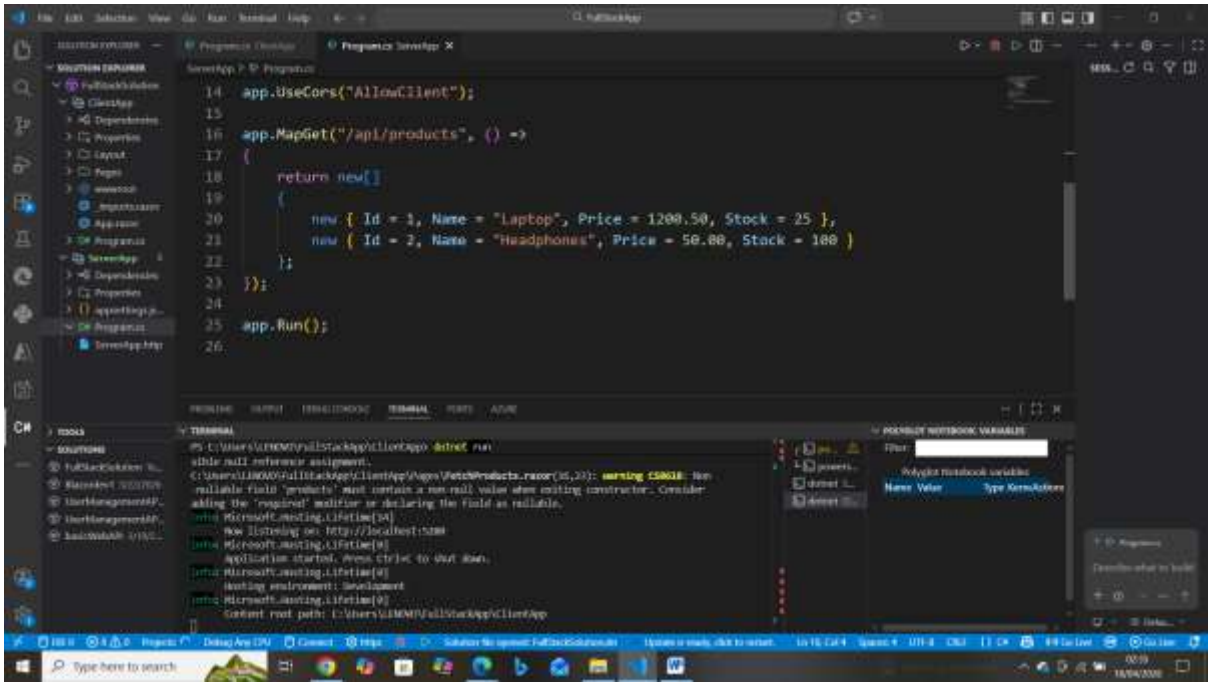
# ServerApp

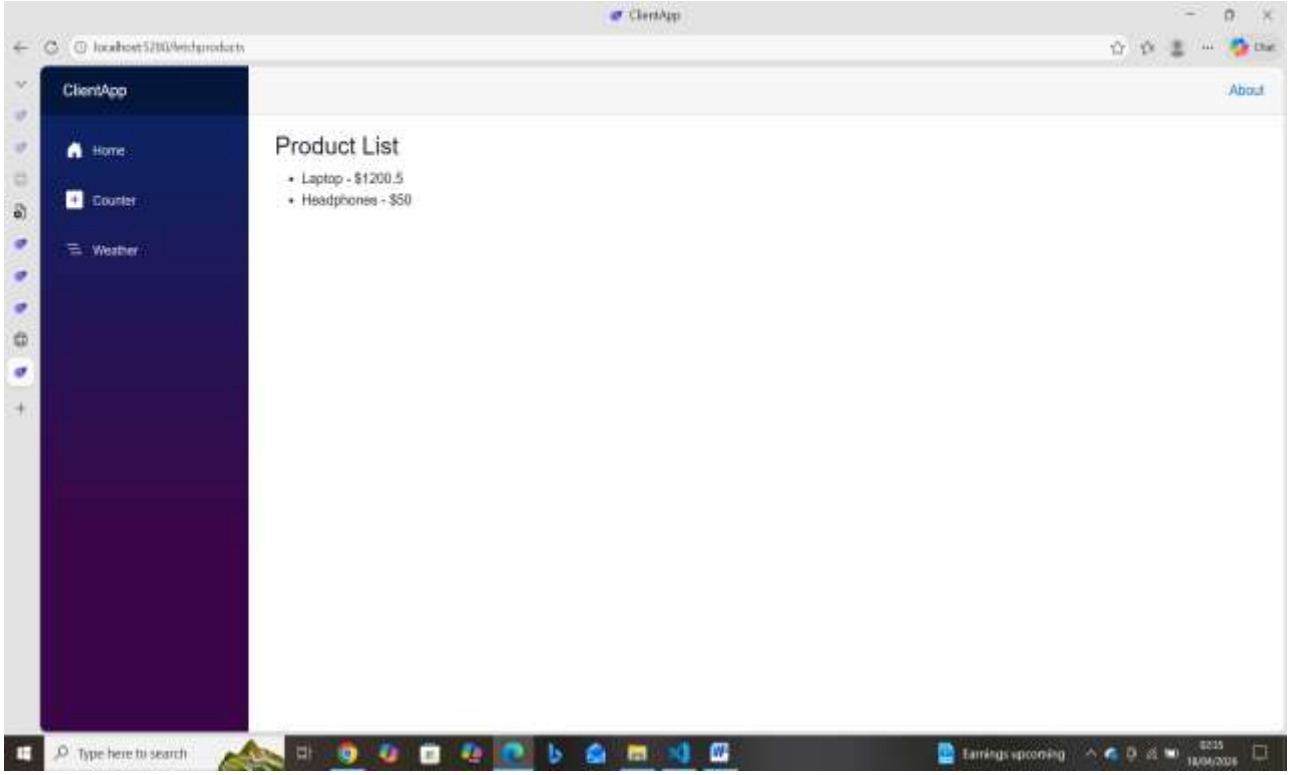
# Program.cs

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowClient",
        policy => policy
            .AllowAnyOrigin()
            .AllowAnyMethod()
            .AllowAnyHeader());
});
var app = builder.Build();
app.UseCors("AllowClient");
app.MapGet("/api/products", () =>
{
    return new[]
    {
        new { Id = 1, Name = "Laptop", Price = 1200.50, Stock = 25 },
        new { Id = 2, Name = "Headphones", Price = 50.00, Stock = 100 }
    };
});
app.Run();
```


Add program arguments
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddCors(options =>
{
options.AddPolicy("AllowClient",
policy => policy
.AllowAnyOrigin()
.AllowAnyMethod()
.AllowAnyHeader());
});
var app = builder.Build();
app.UseCors("AllowClient");


var app = builder.Build();
app.UseCors("AllowClient");
app.MapGet("/api/products", () =>
{
return new[]
{
new { Id = 1, Name = "Laptop", Price = 1200.50, Stock = 25 },
new { Id = 2, Name = "Headphones", Price = 50.00, Stock = 100 }
};
});


app.UseCors("AllowClient");
app.MapGet("/api/products", () =>
{
return new[]
{
new { Id = 1, Name = "Laptop", Price = 1200.50, Stock = 25 },
new { Id = 2, Name = "Headphones", Price = 50.00, Stock = 100 }
};
});
app.Run();


ClientApp
ClientApp
Home
Counter
Weather
About
Product List
Laptop - $1200.5
Headphones - $50

# NESTED JSON

## ServerApp (Program.cs)

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowClient",
        policy => policy
            .AllowAnyOrigin()
            .AllowAnyMethod()
            .AllowAnyHeader());
});
var app = builder.Build();
app.UseCors("AllowClient");
app.MapGet("/api/products", () =>
{
    return new[]
    {
        new
        {
            Id = 1,
            Name = "Laptop",
            Price = 1200.50,
            Stock = 25,
            Category = new { Id = 101, Name = "Electronics" }
        },
        new
        {
            Id = 2,
            Name = "Headphones",
            Price = 50.00,
            Stock = 100,
            Category = new { Id = 102, Name = "Accessories" }
        }
    };
});
app.Run();
```

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowClient",
        policy => policy
            .AllowAnyOrigin()
            .AllowAnyMethod()
            .AllowAnyHeader());
});

var app = builder.Build();

app.UseCors("AllowClient");

app.MapGet("/api/products", () =>
{
    return new[]
    {
        new
        {
            Id = 1,
            Name = "Laptop",
            Price = 1200.50,
            Stock = 25,
            Category = new { Id = 101, Name = "Electronics" }
        },
        new
        {
            Id = 2,
            Name = "Headphones",
            Price = 50.00,
```

```
var app = Builder.Build();

app.UseCors("AllowClient");

app.MapGet("/api/products", () =>
{
    return new[]
    {
        new
        {
            Id = 1,
            Name = "Laptop",
            Price = 1200.50,
            Stock = 25,
            Category = new { Id = 101, Name = "Electronics" }
        },
        new
        {
            Id = 2,
            Name = "Headphones",
            Price = 50.00,
            Stock = 100,
            Category = new { Id = 102, Name = "Accessories" }
        }
    };
});

app.Run();
```

# FetchProducts.razor

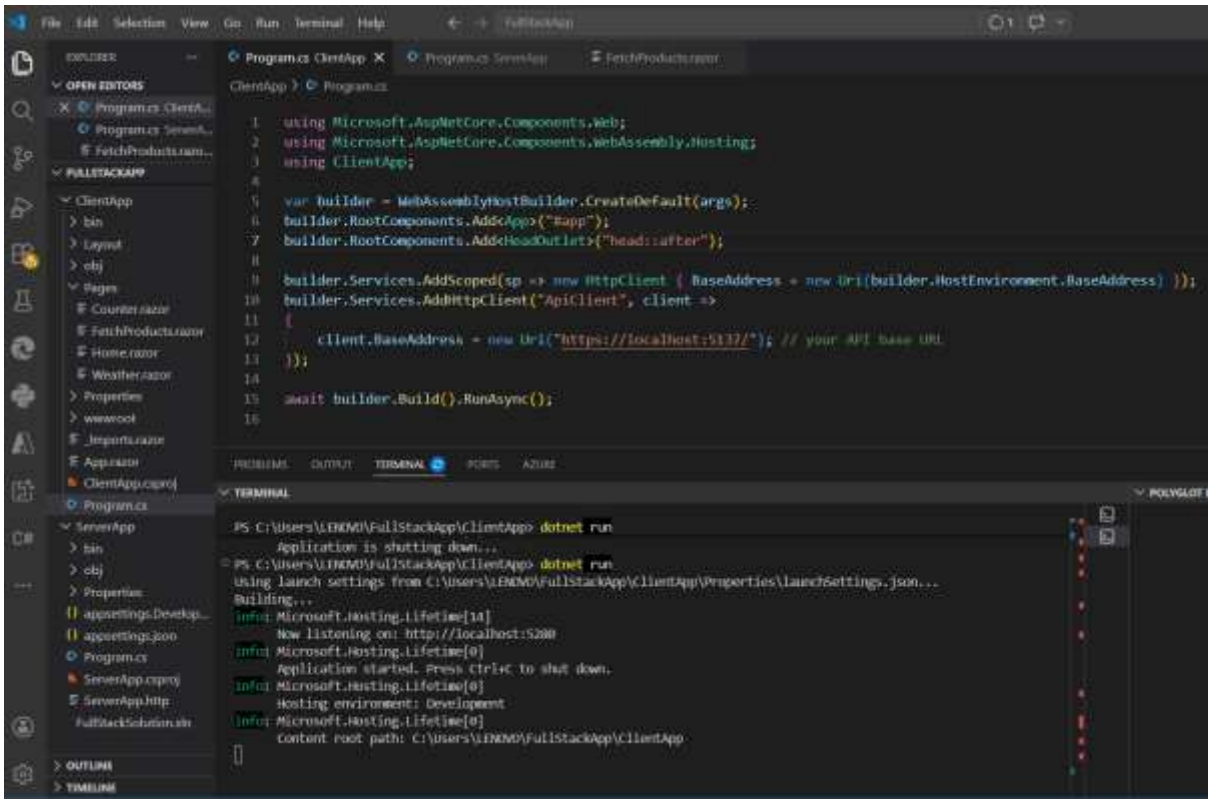
```
@page "/fetchproducts"
@inject HttpClient Http
<h3>Product List</h3>
<ul> @if (products != null)
        {
             foreach(var product in products)
                        {
                <li>Product: @product.Name <br>Price: $@product.Price <br>Qty in
Stock: @product.Stock <br>Category @product.Category?.Name</li>
            }
         }else{
          <li>Loading...</li>
           }
        </ul>
@code{
    private Product[]? products;
     protected override async Task OnInitializedAsync()
      {
        // API call logic will go here
         products = await
Http.GetFromJsonAsync<Product[]>("http://localhost:5137/api/products");
      }
    public class Product
     {
    public int Id { get; set; }
    public string? Name { get; set; }
    public double Price { get; set; }
    public int Stock { get; set; }
    public Category? Category { get; set; }
     }
     public class Category
   {
```

```
    public int Id { get; set; }
    public string? Name { get; set; }
}
}
```

```
@page "/Fetchproducts"
@inject HttpClient Http
<h3>Product List</h3>
<ul> @if (products != null)
        {
            foreach(var product in products)
                {
                <li>Product: @product.Name <br>Price: $@product.Price <br>Qty in Stock: @product.Stock <br>Category @
            }
        }else{
         <li>Loading...</li>
         }
        </ul>

@code{
    private Product[]? products;
    protected override async Task OnInitializedAsync()
     {
       // API call logic will go here
        products = await Http.GetFromJsonAsync<Product[]>("http://localhost:5137/api/products");
     }
    public class Product
     {
    public int Id { get; set; }
    public string? Name { get; set; }
    public double Price { get; set; }
    public int Stock { get; set; }
    public Category? Category { get; set; }
     }
     public class Category
    {
```

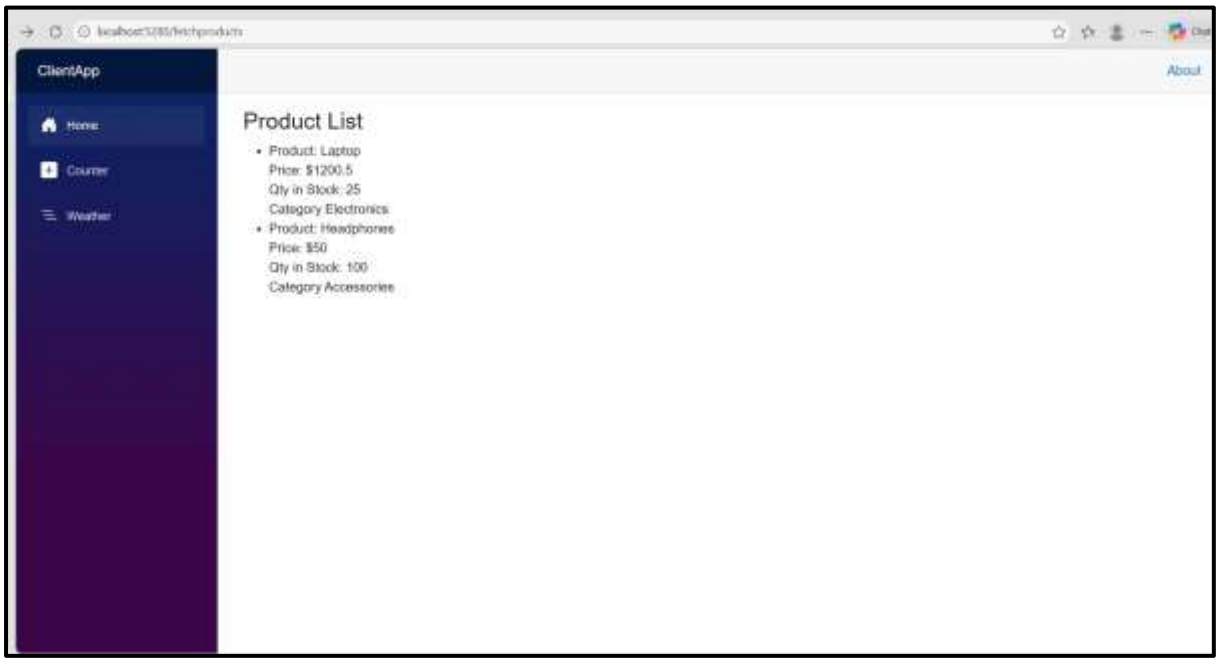
```
            }
        }else{
         <li>Loading...</li>
         }
        </ul>

@code{
    private Product[]? products;
    protected override async Task OnInitializedAsync()
     {
       // API call logic will go here
        products = await Http.GetFromJsonAsync<Product[]>("http://localhost:5137/api/products");
     }
    public class Product
     {
    public int Id { get; set; }
    public string? Name { get; set; }
    public double Price { get; set; }
    public int Stock { get; set; }
    public Category? Category { get; set; }
     }
     public class Category
    {
    public int Id { get; set; }
    public string? Name { get; set; }
}
}
```

# ClientApp (Program.cs)




```
using Microsoft.AspNetCore.Components.Web;
using Microsoft.AspNetCore.Components.WebAssembly.Hosting;
using ClientApp;
var builder = WebAssemblyHostBuilder.CreateDefault(args);
builder.RootComponents.Add<App>("#app");
builder.RootComponents.Add<HeadOutlet>("head::after");

builder.Services.AddScoped(sp => new HttpClient { BaseAddress = new
Uri(builder.HostEnvironment.BaseAddress) });
builder.Services.AddHttpClient("ApiClient", client =>
{
    client.BaseAddress = new Uri("https://localhost:5137/"); // your API base URL
});
await builder.Build().RunAsync();
```

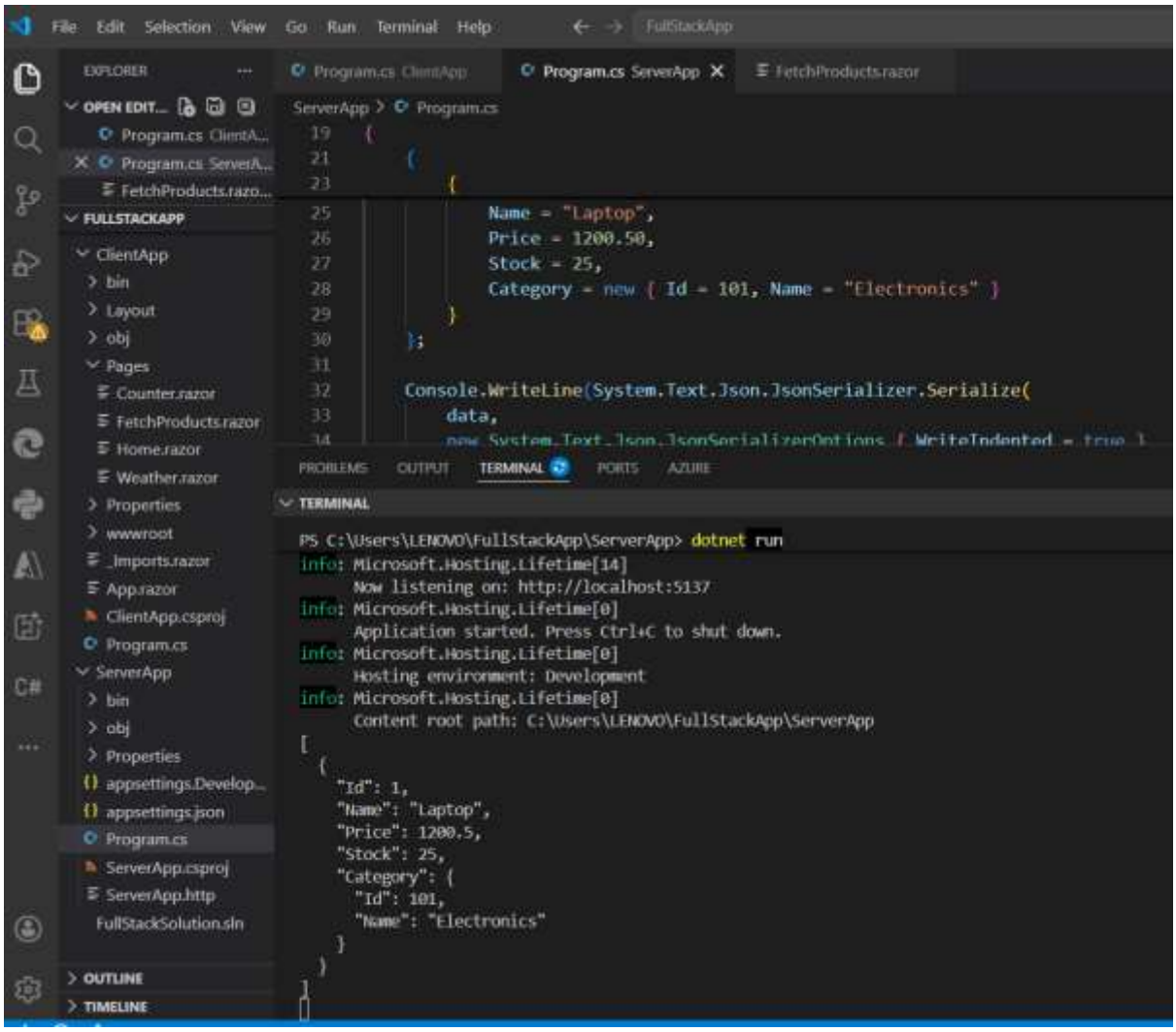
# OUTPUT



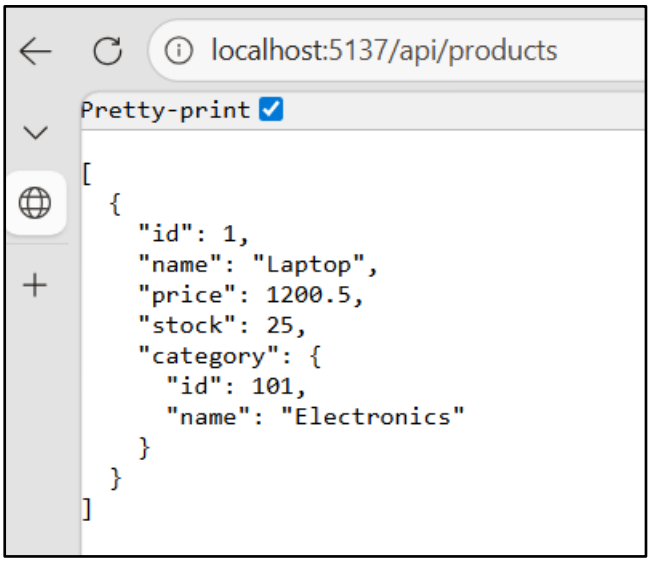

# PRINT OR SERIALIZE TO CONSOLE

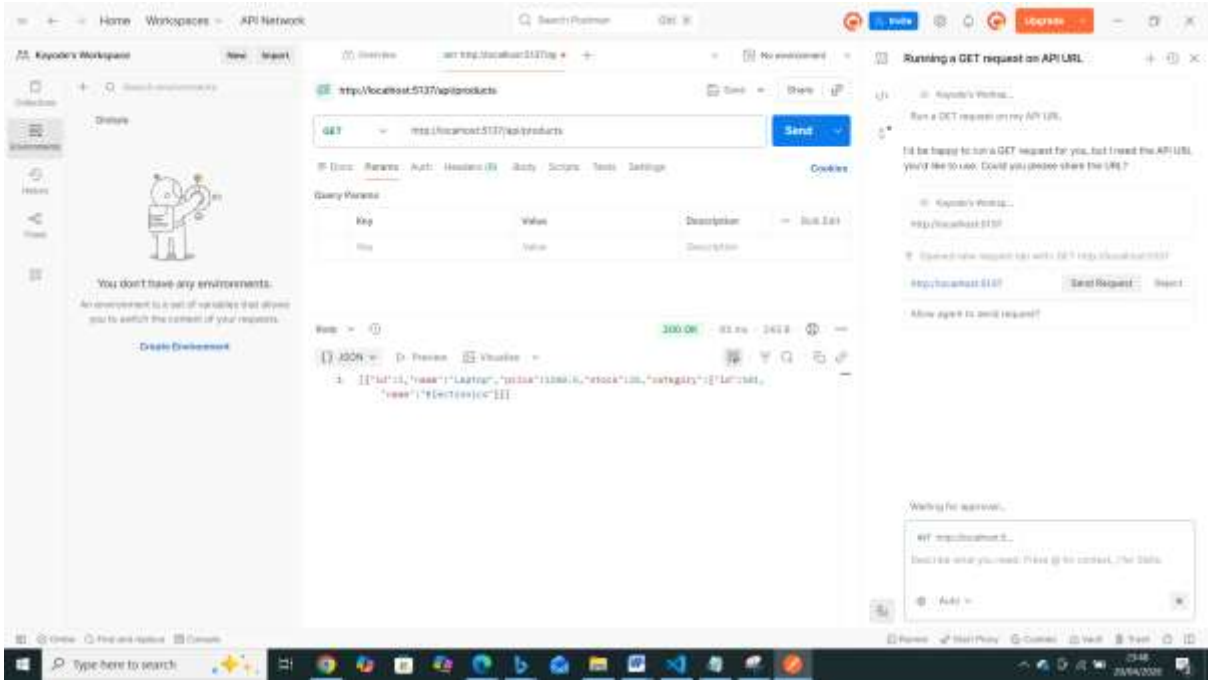
```
using System.Text.Json;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowClient",
        policy => policy
            .AllowAnyOrigin()
            .AllowAnyMethod()
            .AllowAnyHeader());
});

var app = builder.Build();   // ⚠ MUST come before using app

app.UseCors("AllowClient");

app.MapGet("/api/products", () =>
{
    var data = new[]
    {
        new
```

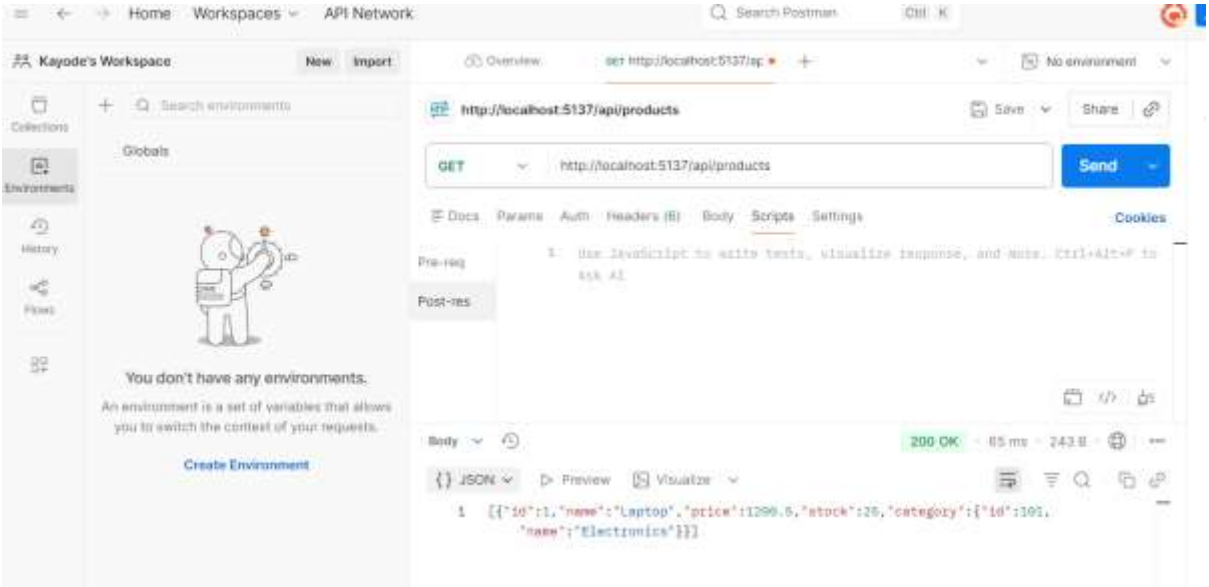
```
        {
            Id = 1,
            Name = "Laptop",
            Price = 1200.50,
            Stock = 25,
            Category = new { Id = 101, Name = "Electronics" }
        }
    };

    Console.WriteLine(System.Text.Json.JsonSerializer.Serialize(
        data,
        new System.Text.Json.JsonSerializerOptions { WriteIndented = true }
    ));

    return data;
});

app.Run();
```

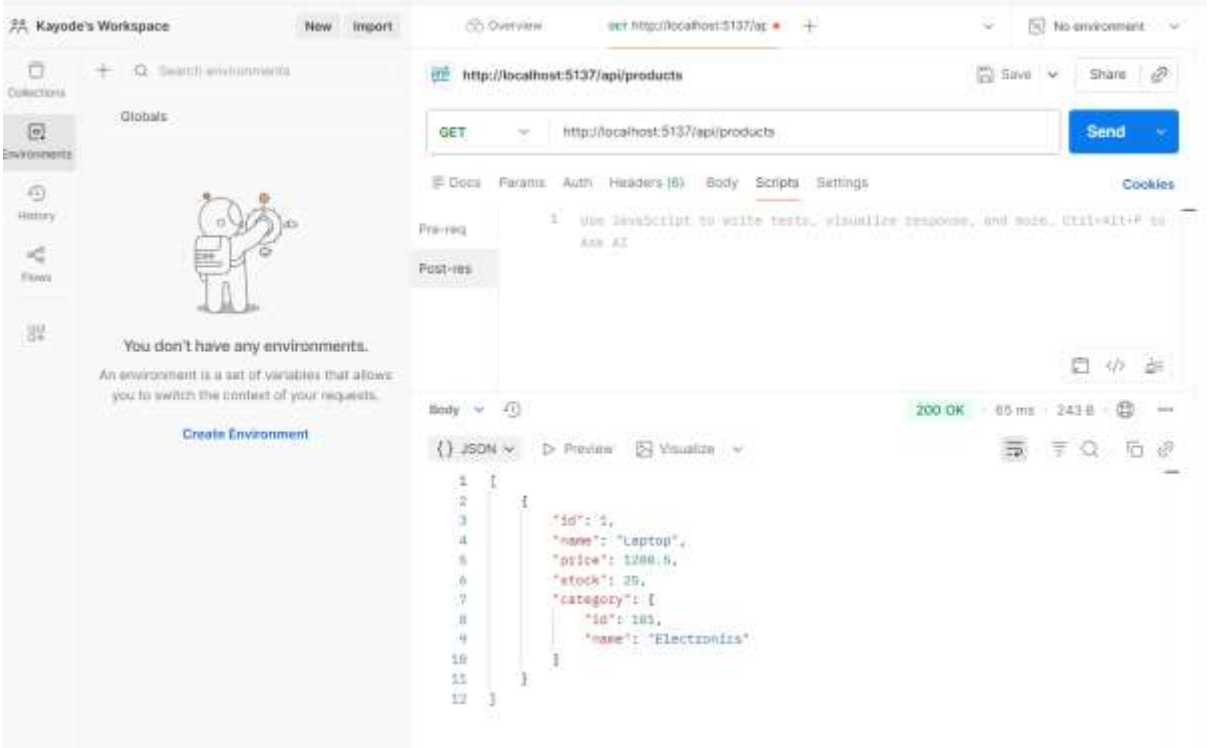


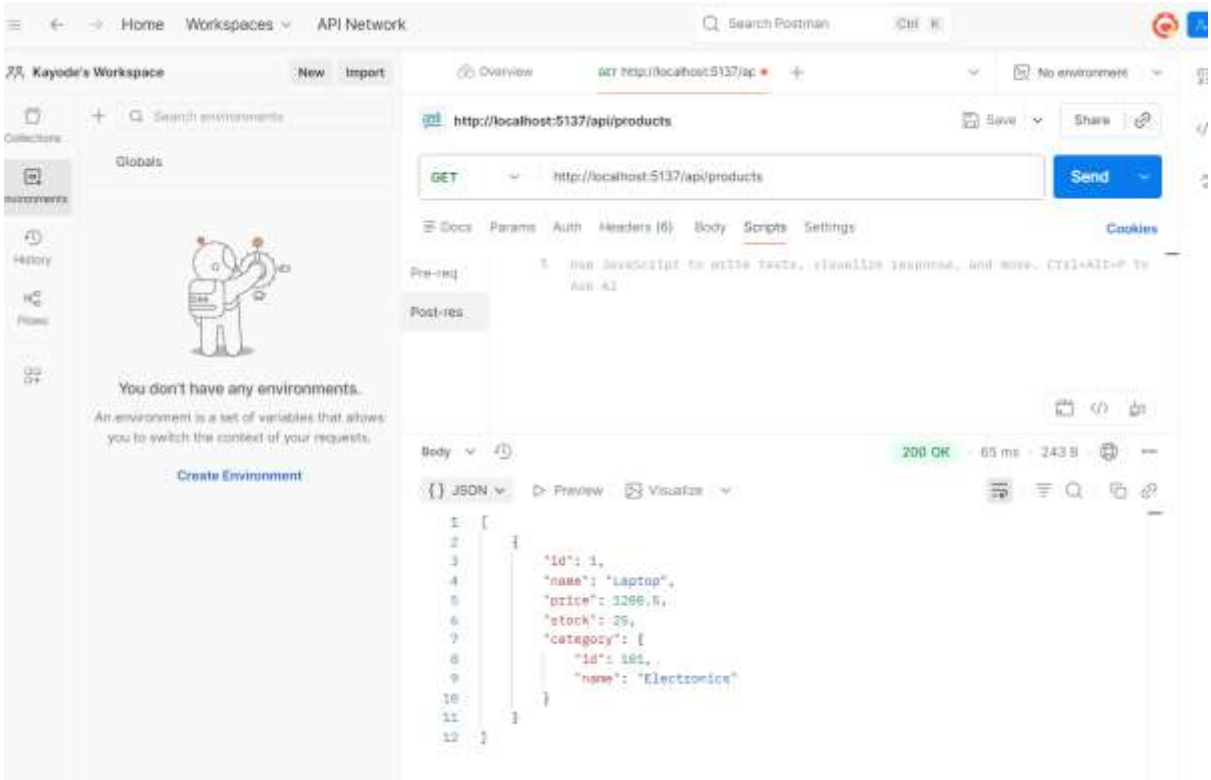
Serializing the products using postman

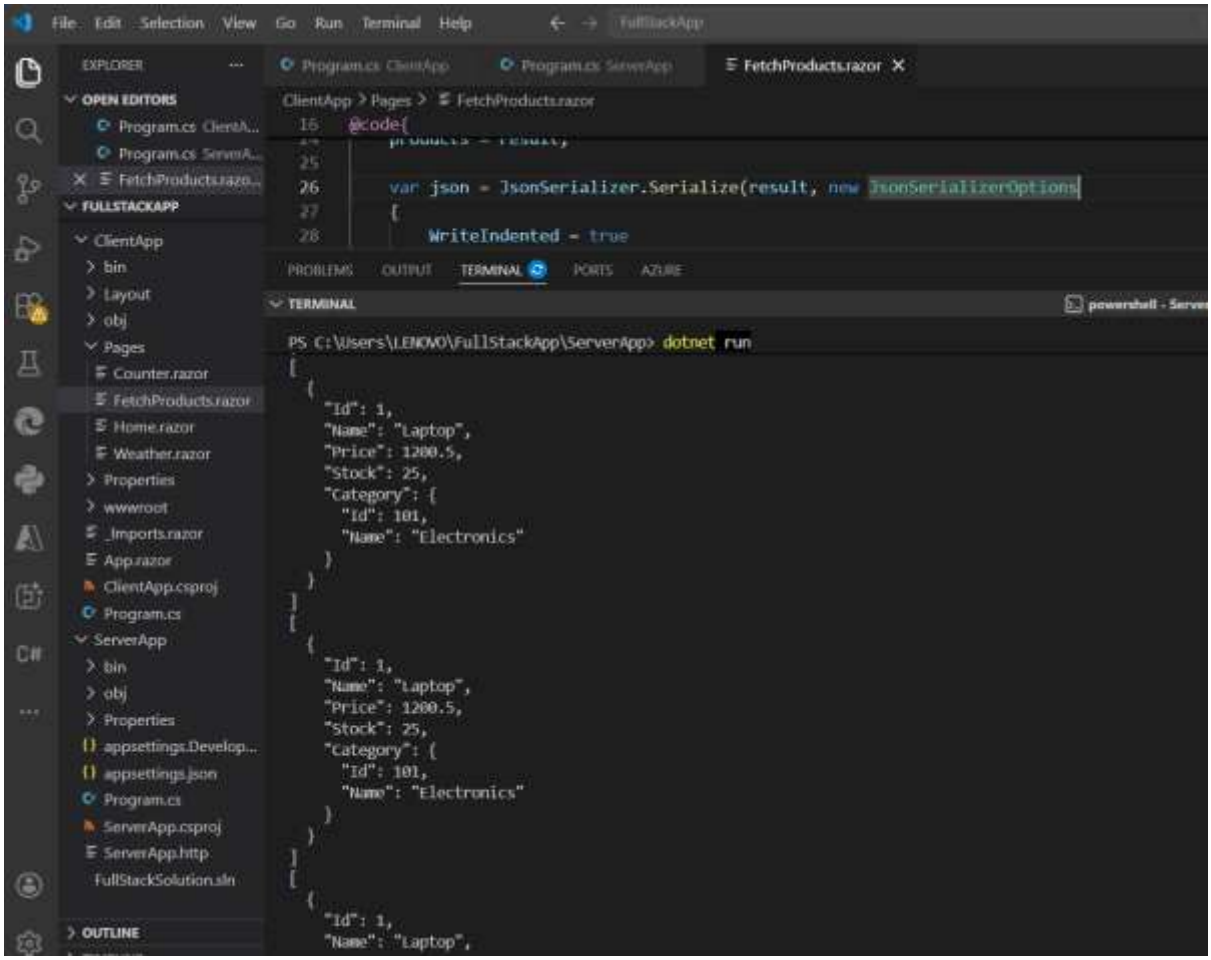



Home
Workspaces
API Network
Search Postman
Ctrl K
Kayode's Workspace
New
Import
Overview
get http://localhost:5137/ap
No environment
Collections
Environments
History
Flows
Search environments
Globals
You don't have any environments.
An environment is a set of variables that allows you to switch the context of your requests.
Create Environment
http://localhost:5137/api/products
Save
Share
GET
http://localhost:5137/api/products
Send
Docs
Params
Auth
Headers (8)
Body
Scripts
Settings
Cookies
Pre-req
Post-res
Body
200 OK
65 ms
243 B
JSON
Preview
Visualize
[{"id":1,"name":"Laptop","price":1200.5,"stock":20,"category":{"id":101,"name":"Electronics"}}]

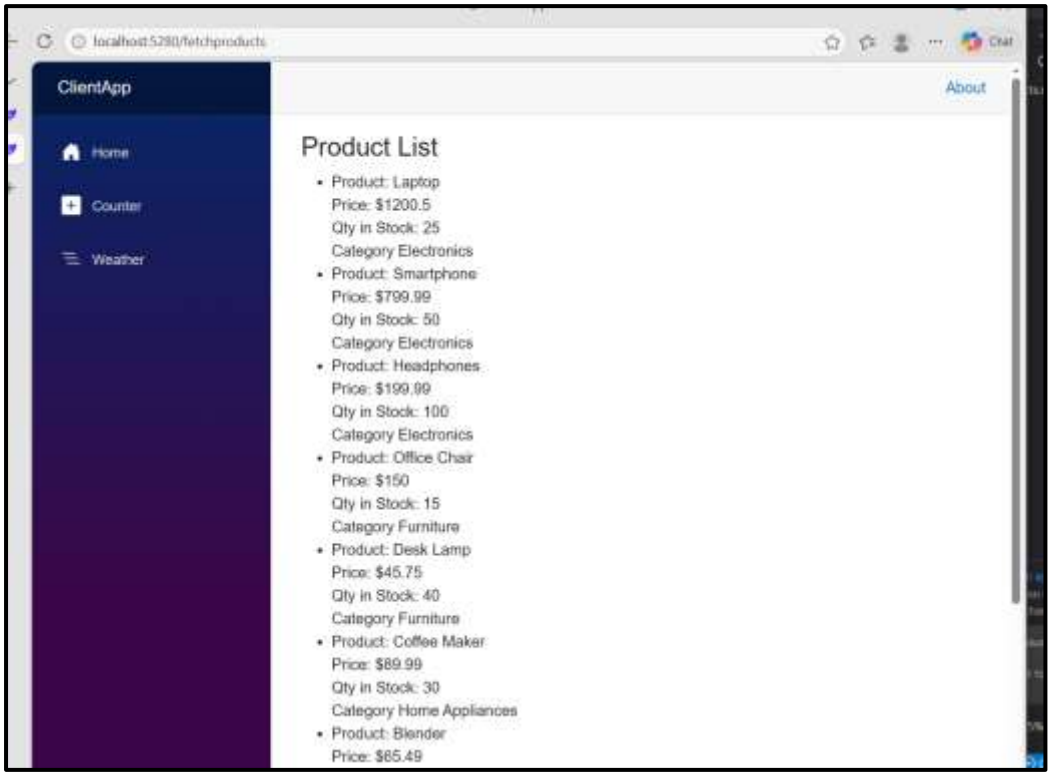

Kayode's Workspace
New
Import
Overview
get http://localhost:5137/ap
No environment
Collections
Environments
History
Flows
Search environments
Globals
You don't have any environments.
An environment is a set of variables that allows you to switch the context of your requests.
Create Environment
http://localhost:5137/api/products
Save
Share
GET
http://localhost:5137/api/products
Send
Docs
Params
Auth
Headers (8)
Body
Scripts
Settings
Cookies
Pre-req
Post-res
Body
200 OK
65 ms
243 B
JSON
Preview
Visualize
[
{
"id": 1,
"name": "Laptop",
"price": 1200.5,
"stock": 20,
"category": {
"id": 101,
"name": "Electronics"
}
}
]


Home
Workspaces
API Network
Search Postman
Kayode's Workspace
New
Import
Overview
No environment
Globals
You don't have any environments.
An environment is a set of variables that allows you to switch the context of your requests.
Create Environment
http://localhost:5137/api/products
Save
Share
GET
http://localhost:5137/api/products
Send
Docs
Params
Auth
Headers (6)
Body
Scripts
Settings
Cookies
Pre-req.
Post-res.
Body
200 OK
65 ms
243 B
JSON
Preview
Visualize
[
{
"Id": 1,
"name": "Laptop",
"price": 1200.5,
"stock": 25,
"category": {
"Id": 101,
"name": "Electronics"
}
}
]

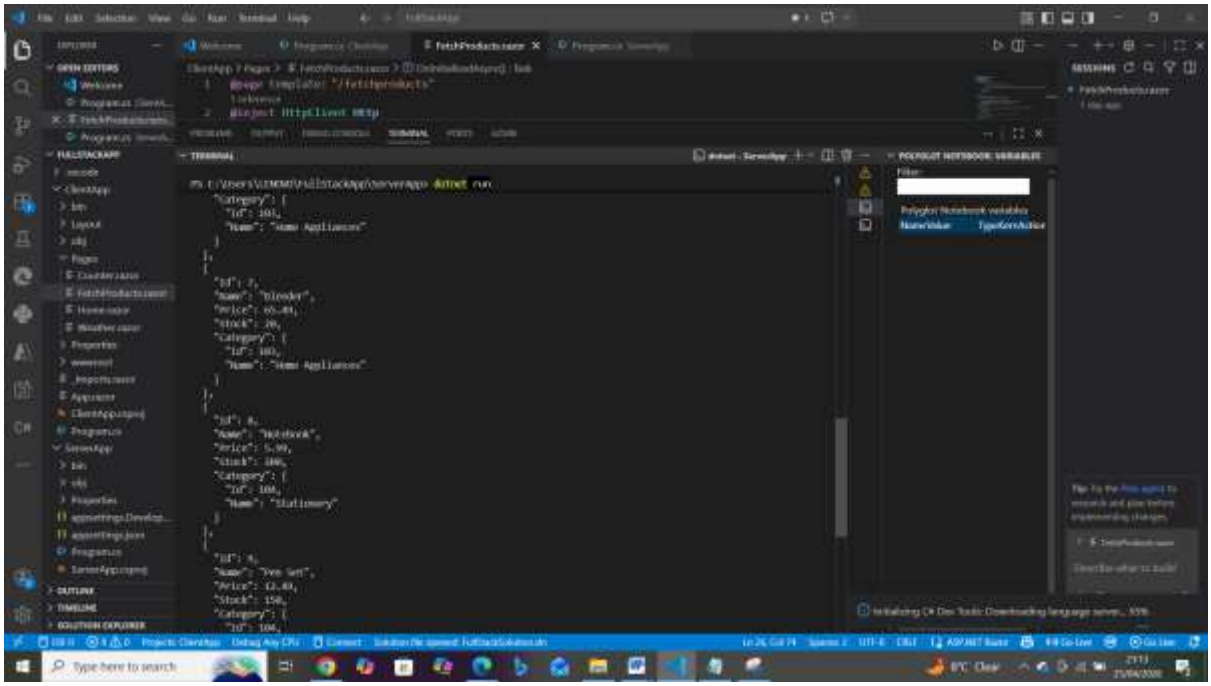

File
Edit
Selection
View
Go
Run
Terminal
Help
FullStackApp
EXPLORER
OPEN EDITORS
Program.cs ClientApp
Program.cs ServerApp
FetchProducts.razor
FULLSTACKAPP
ClientApp
bin
Layout
obj
Pages
Counter.razor
FetchProducts.razor
Home.razor
Weather.razor
Properties
wwwroot
_Imports.razor
App.razor
ClientApp.csproj
Program.cs
ServerApp
bin
obj
Properties
appsettings.Develop...
appsettings.json
Program.cs
ServerApp.csproj
ServerApp.http
FullStackSolution.sln
OUTLINE
ClientApp > Pages > FetchProducts.razor
@code{
var json = JsonSerializer.Serialize(result, new JsonSerializerOptions
{
WriteIndented = true
PROBLEMS
OUTPUT
TERMINAL
PORTS
AZURE
TERMINAL
powershell - Server
PS C:\Users\LENOVO\FullStackApp\ServerApp> dotnet run
[
{
"Id": 1,
"Name": "Laptop",
"Price": 1200.5,
"Stock": 25,
"Category": {
"Id": 101,
"Name": "Electronics"
}
}
]
[
{
"Id": 1,
"Name": "Laptop",
"Price": 1200.5,
"Stock": 25,
"Category": {
"Id": 101,
"Name": "Electronics"
}
}
]
[
{
"Id": 1,
"Name": "Laptop",

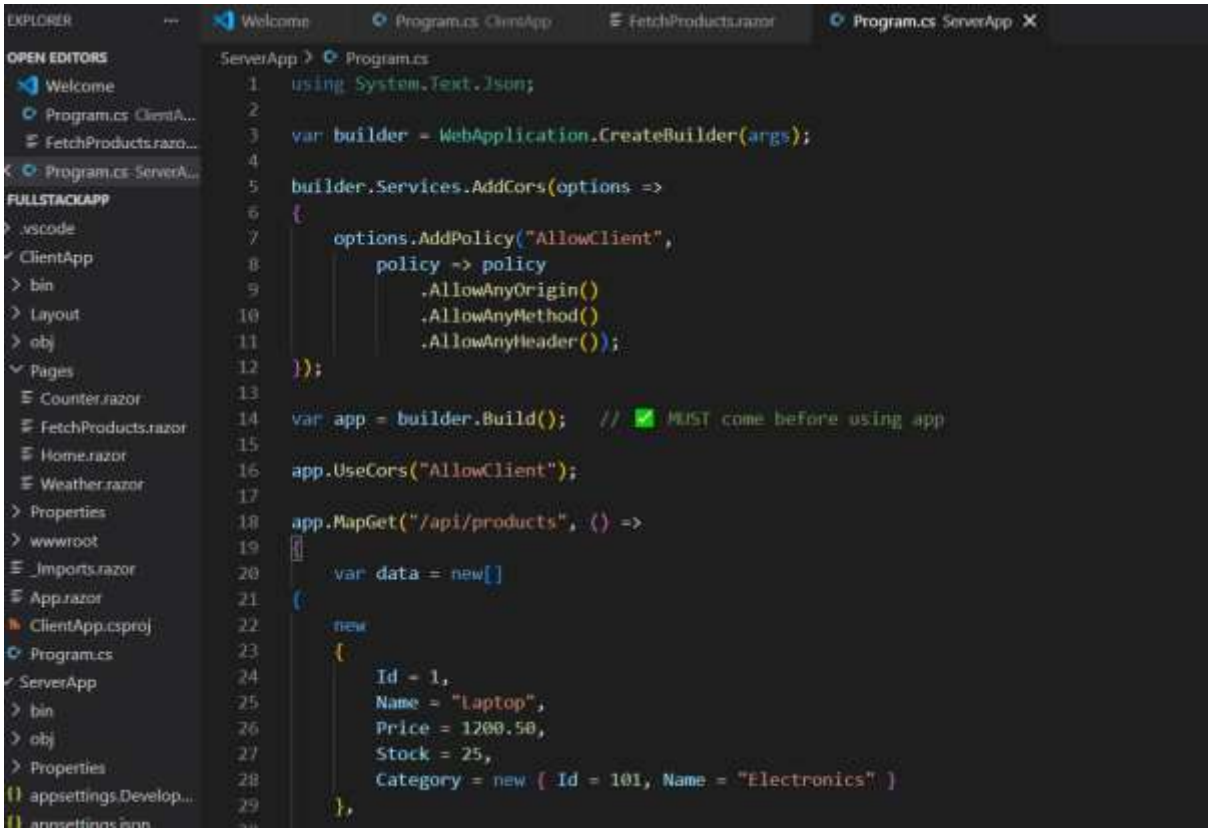
# EXTENDED FUNCTIONALITY TO ACCOMMODATE MORE DATA

## Blazor Frontend

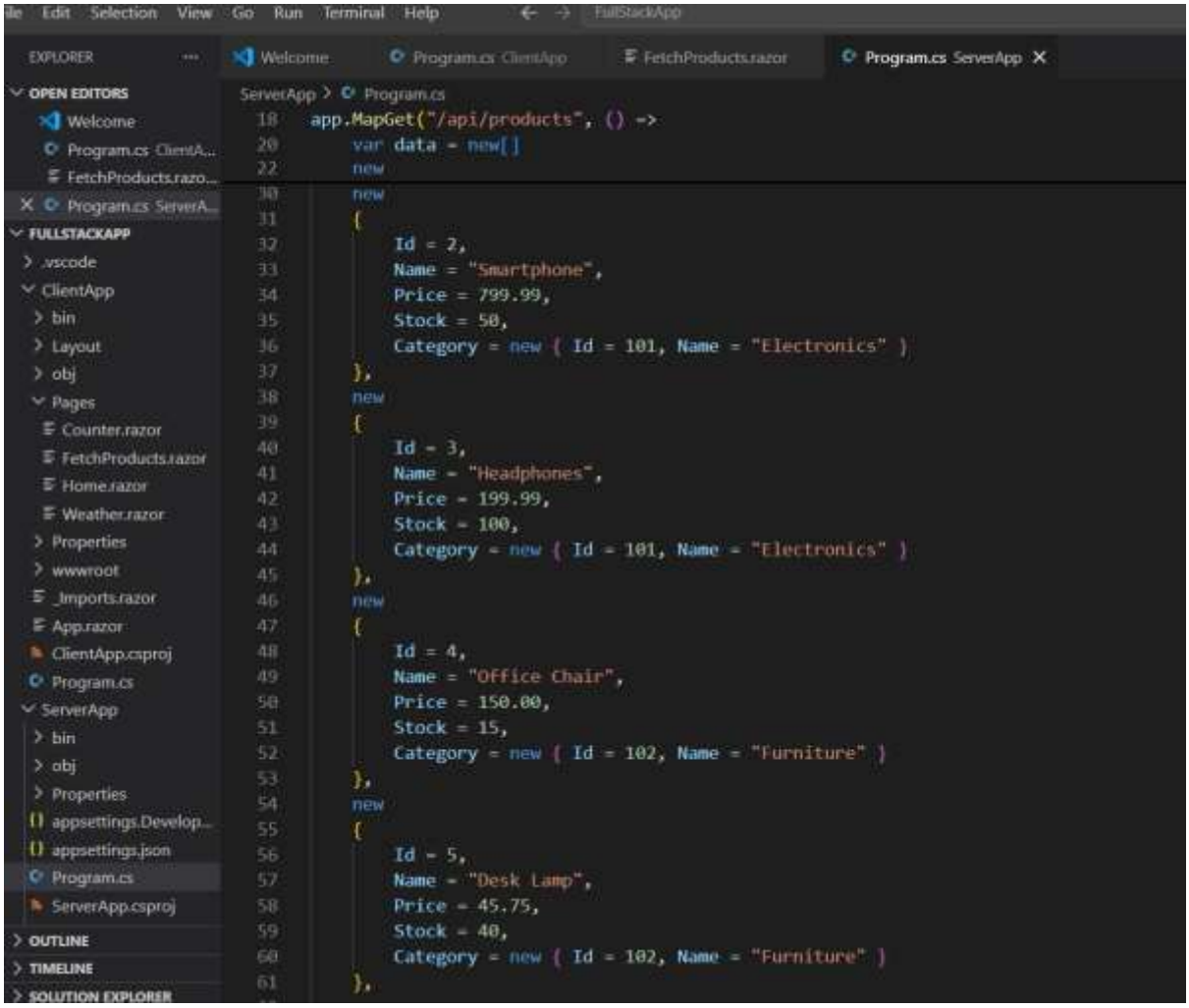


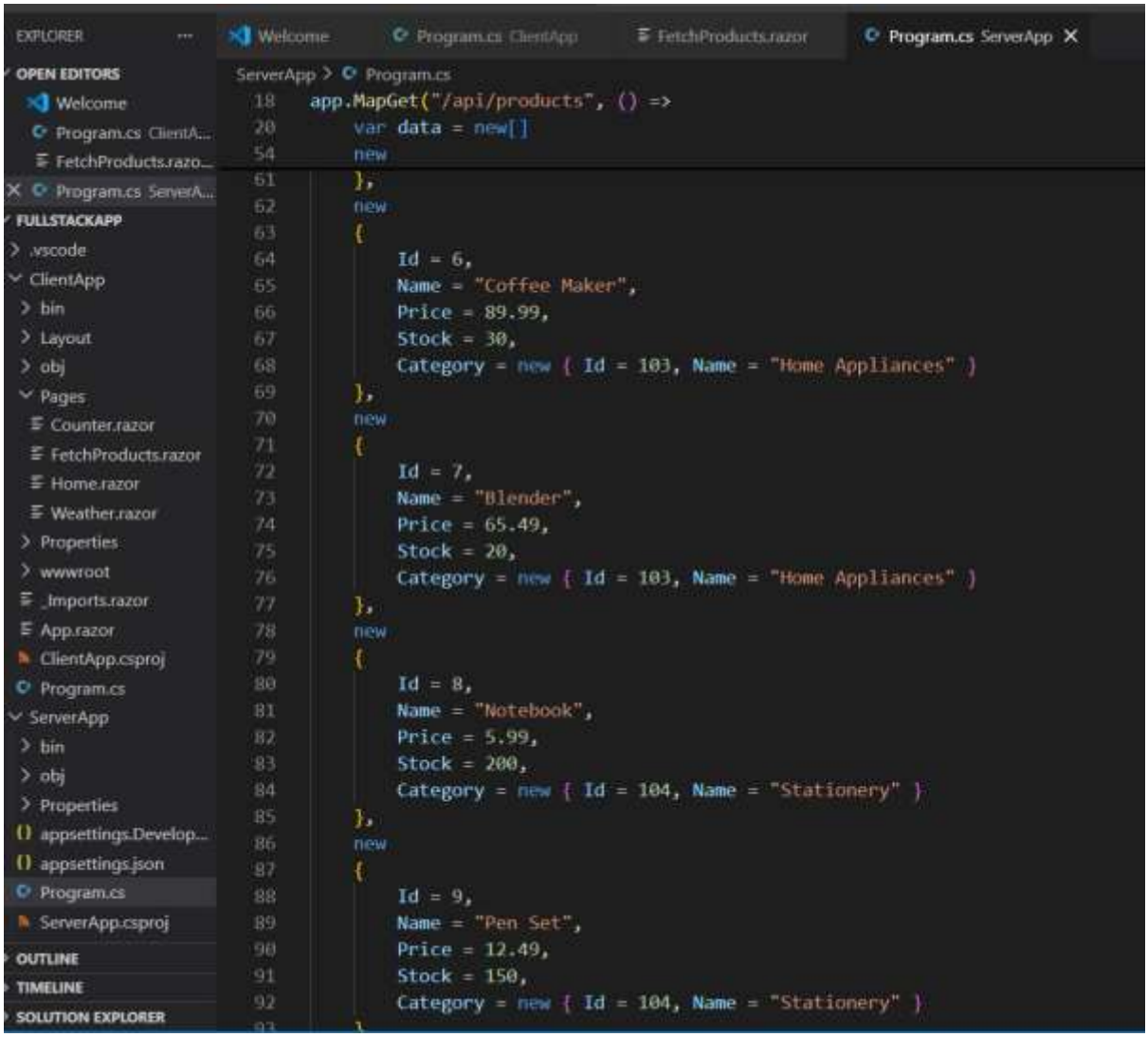

Backend Console Display





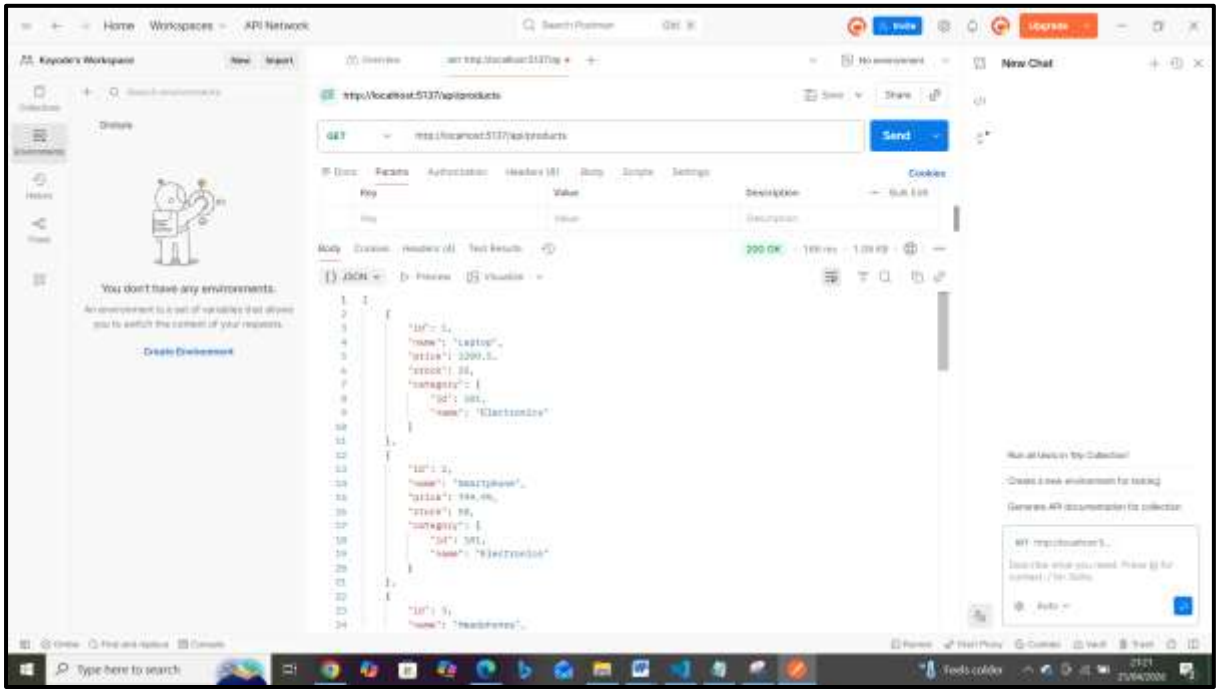

The following programs show debugged generated codes for creating the endpoints in the server.


Edit Selection View Go Run Terminal Help
FullStackApp
EXPLORER
Welcome
Program.cs ClientApp
FetchProducts.razor
Program.cs ServerApp
OPEN EDITORS
Welcome
Program.cs ClientA...
FetchProducts.razo...
Program.cs ServerA...
FULLSTACKAPP
.vscode
ClientApp
bin
Layout
obj
Pages
Counter.razor
FetchProducts.razor
Home.razor
Weather.razor
Properties
wwwroot
_Imports.razor
App.razor
ClientApp.csproj
Program.cs
ServerApp
bin
obj
Properties
appsettings.Develop...
appsettings.json
Program.cs
ServerApp.csproj
OUTLINE
TIMELINE
SOLUTION EXPLORER
ServerApp > Program.cs
18 app.MapGet("/api/products", () =>
20 var data = new[]
22 new
30 new
31 {
32 Id = 2,
33 Name = "Smartphone",
34 Price = 799.99,
35 Stock = 50,
36 Category = new { Id = 101, Name = "Electronics" }
37 },
38 new
39 {
40 Id = 3,
41 Name = "Headphones",
42 Price = 199.99,
43 Stock = 100,
44 Category = new { Id = 101, Name = "Electronics" }
45 },
46 new
47 {
48 Id = 4,
49 Name = "Office Chair",
50 Price = 150.00,
51 Stock = 15,
52 Category = new { Id = 102, Name = "Furniture" }
53 },
54 new
55 {
56 Id = 5,
57 Name = "Desk Lamp",
58 Price = 45.75,
59 Stock = 40,
60 Category = new { Id = 102, Name = "Furniture" }
61 },


EXPLORER
Welcome
Program.cs ClientApp
FetchProducts.razor
Program.cs ServerApp
OPEN EDITORS
Welcome
Program.cs ClientA...
FetchProducts.razo...
Program.cs ServerA...
FULLSTACKAPP
.vscode
ClientApp
bin
Layout
obj
Pages
Counter.razor
FetchProducts.razor
Home.razor
Weather.razor
Properties
wwwroot
_Imports.razor
App.razor
ClientApp.csproj
Program.cs
ServerApp
bin
obj
Properties
appsettings.Develop...
appsettings.json
Program.cs
ServerApp.csproj
OUTLINE
TIMELINE
SOLUTION EXPLORER
ServerApp > Program.cs
18 app.MapGet("/api/products", () =>
20 var data = new[]
54 new
61 },
62 new
63 {
64 Id = 6,
65 Name = "Coffee Maker",
66 Price = 89.99,
67 Stock = 30,
68 Category = new { Id = 103, Name = "Home Appliances" }
69 },
70 new
71 {
72 Id = 7,
73 Name = "Blender",
74 Price = 65.49,
75 Stock = 20,
76 Category = new { Id = 103, Name = "Home Appliances" }
77 },
78 new
79 {
80 Id = 8,
81 Name = "Notebook",
82 Price = 5.99,
83 Stock = 200,
84 Category = new { Id = 104, Name = "Stationery" }
85 },
86 new
87 {
88 Id = 9,
89 Name = "Pen Set",
90 Price = 12.49,
91 Stock = 150,
92 Category = new { Id = 104, Name = "Stationery" }

```
app.MapGet("/api/products", () =>
    var data = new[]
    new
        Name = "Notebook",
        Price = 5.99,
        Stock = 200,
        Category = new { Id = 104, Name = "Stationery" }
    },
    new
    {
        Id = 9,
        Name = "Pen Set",
        Price = 12.49,
        Stock = 150,
        Category = new { Id = 104, Name = "Stationery" }
    },
    new
    {
        Id = 10,
        Name = "Backpack",
        Price = 39.99,
        Stock = 60,
        Category = new { Id = 105, Name = "Accessories" }
    }
};

    Console.WriteLine(System.Text.Json.JsonSerializer.Serialize(
        data,
        new System.Text.Json.JsonSerializerOptions { WriteIndented = true }
    ));

    return data;
});

app.Run();
```

Testing the API call from the server using the postman



http://localhost:5137/api/products

Save Share

GET http://localhost:5137/api/products Send

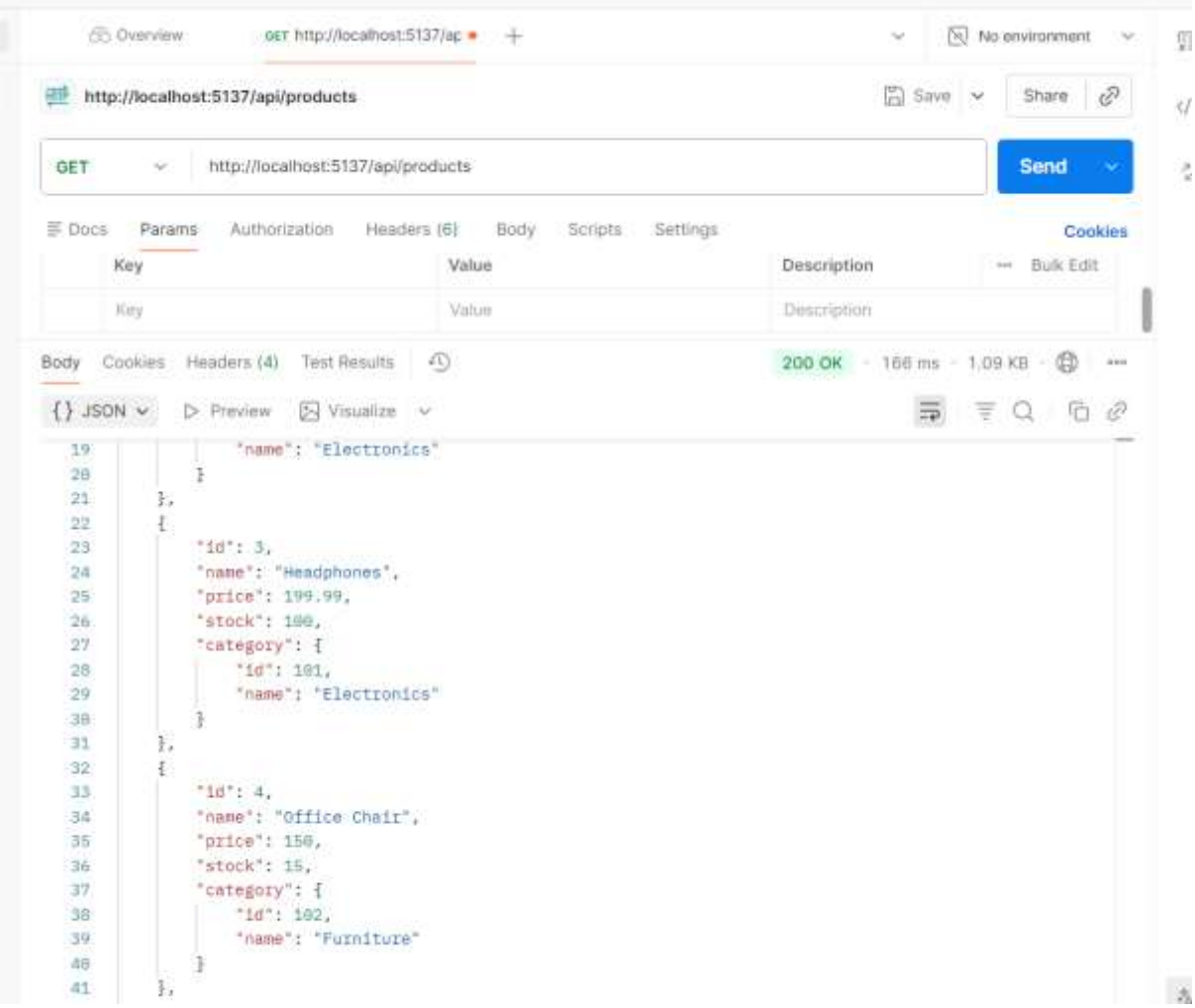
Docs Params Authorization Headers (6) Body Scripts Settings Cookies

| Key | Value | Description | Bulk Edit |
|---|---|---|---|
| Key | Value | Description | |

Body Cookies Headers (4) Test Results 200 OK 166 ms 1.09 KB

{} JSON Preview Visualize


```
[
    {
        "id": 1,
        "name": "Laptop",
        "price": 1200.5,
        "stock": 25,
        "category": {
            "id": 101,
            "name": "Electronics"
        }
    },
    {
        "id": 2,
        "name": "Smartphone",
        "price": 799.99,
        "stock": 50,
        "category": {
            "id": 101,
            "name": "Electronics"
        }
    },
    {
```

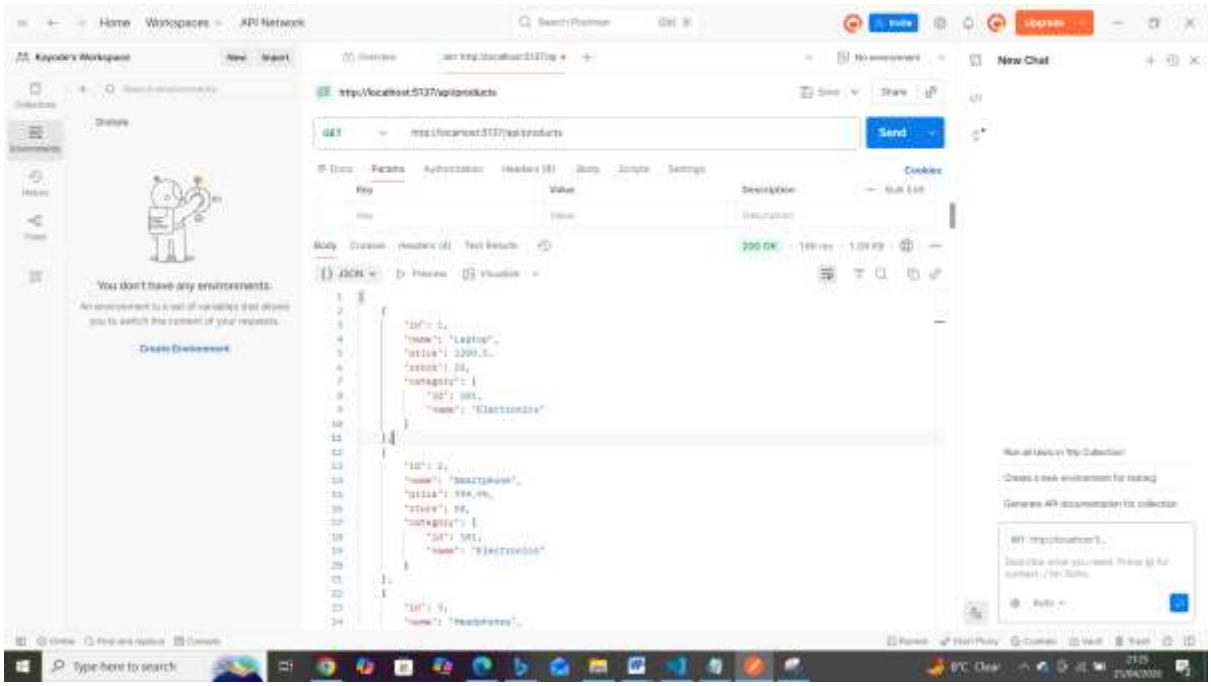




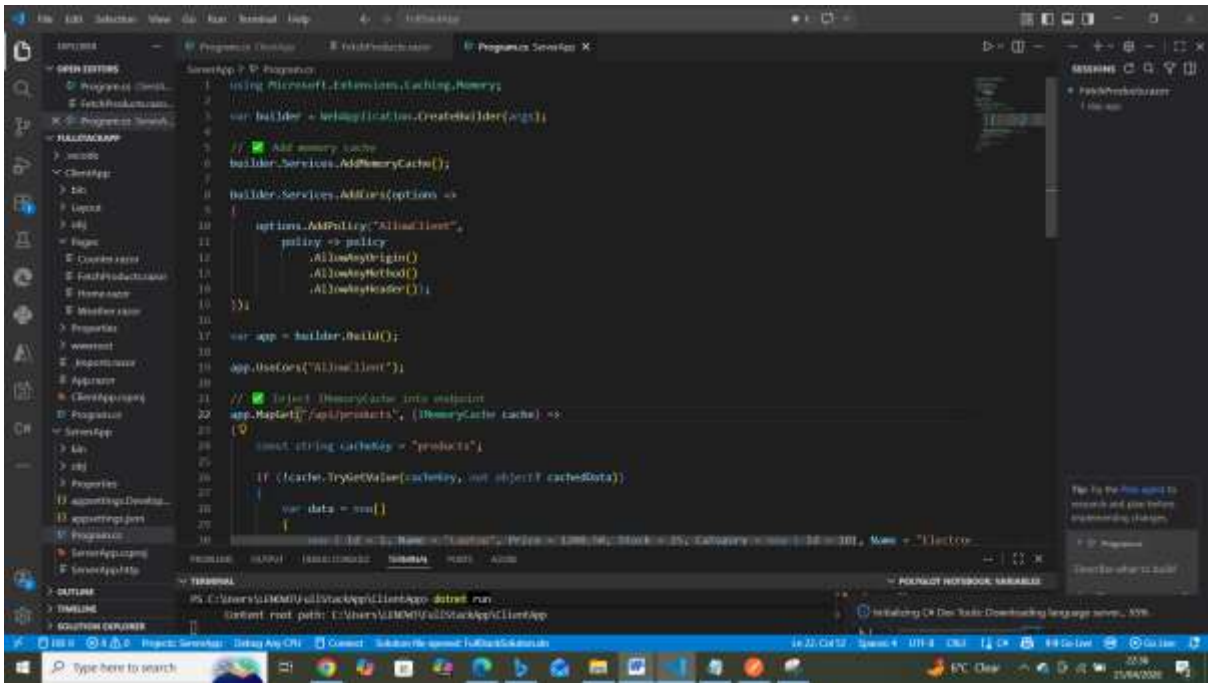
```
[
    {
        "id": 1,
        "name": "Laptop",
        "price": 1200.5,
        "stock": 25,
        "category": {
            "id": 101,
            "name": "Electronics"
        }
    },
    {
        "id": 2,
        "name": "Smartphone",
        "price": 799.99,
        "stock": 50,
        "category": {
            "id": 101,
            "name": "Electronics"
        }
```

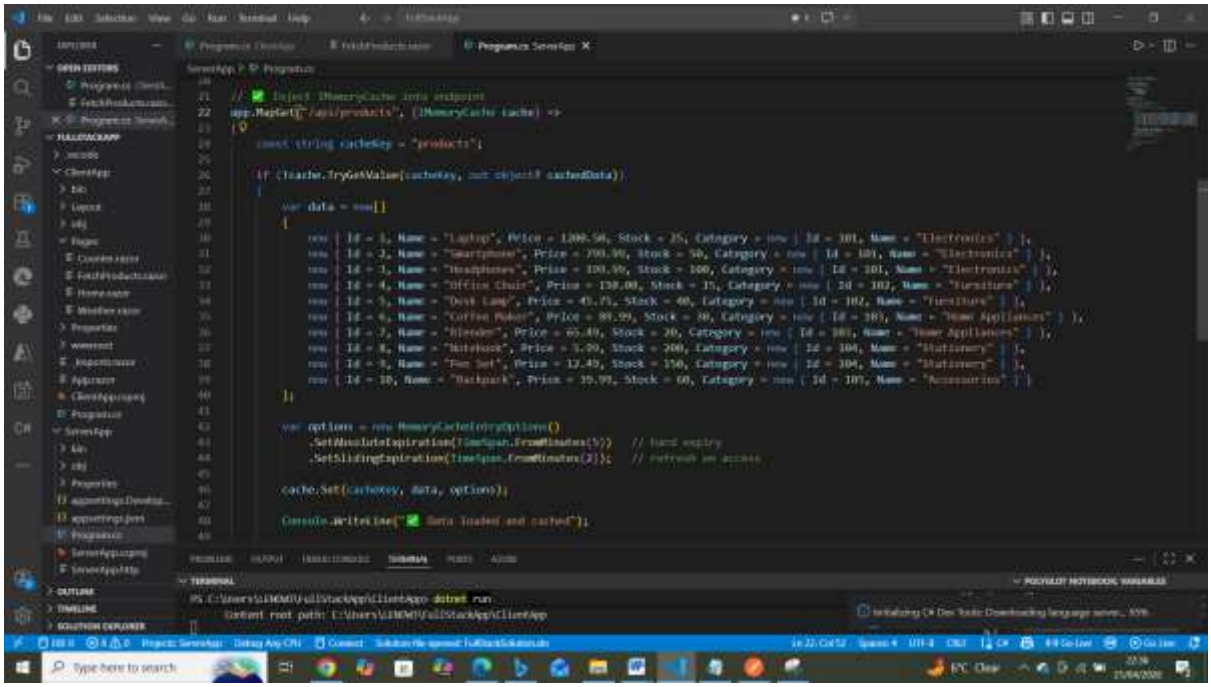
```
},
{
    "id": 3,
    "name": "Headphones",
    "price": 199.99,
    "stock": 100,
    "category": {
        "id": 101,
        "name": "Electronics"
    }
},
{
    "id": 4,
    "name": "Office Chair",
    "price": 150,
    "stock": 15,
    "category": {
        "id": 102,
        "name": "Furniture"
    }
},
{
    "id": 5,
    "name": "Desk Lamp",
    "price": 45.75,
    "stock": 40,
    "category": {
        "id": 102,
        "name": "Furniture"
    }
},
{
    "id": 6,
    "name": "Coffee Maker",
    "price": 89.99,
    "stock": 30,
    "category": {
        "id": 103,
        "name": "Home Appliances"
    }
},
{
    "id": 7,
    "name": "Blender",
    "price": 65.49,
    "stock": 20,
    "category": {
        "id": 103,
        "name": "Home Appliances"
    }
},
```

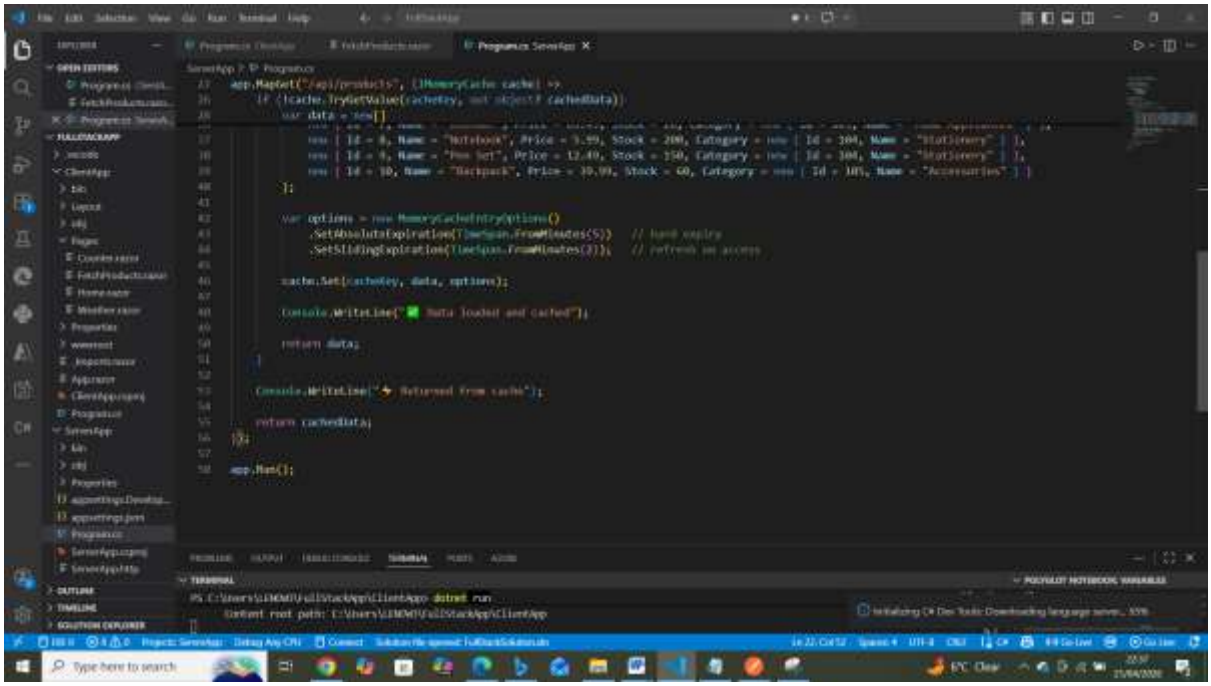
```
    {
        "id": 8,
        "name": "Notebook",
        "price": 5.99,
        "stock": 200,
        "category": {
            "id": 104,
            "name": "Stationery"
        }
    },
    {
        "id": 9,
        "name": "Pen Set",
        "price": 12.49,
        "stock": 150,
        "category": {
            "id": 104,
            "name": "Stationery"
        }
    },
    {
        "id": 10,
        "name": "Backpack",
        "price": 39.99,
        "stock": 60,
        "category": {
            "id": 105,
            "name": "Accessories"
        }
    }
]
```

IMPLEMENTING CACHING AND REFACTORING TO INCLUDE COMMENTS





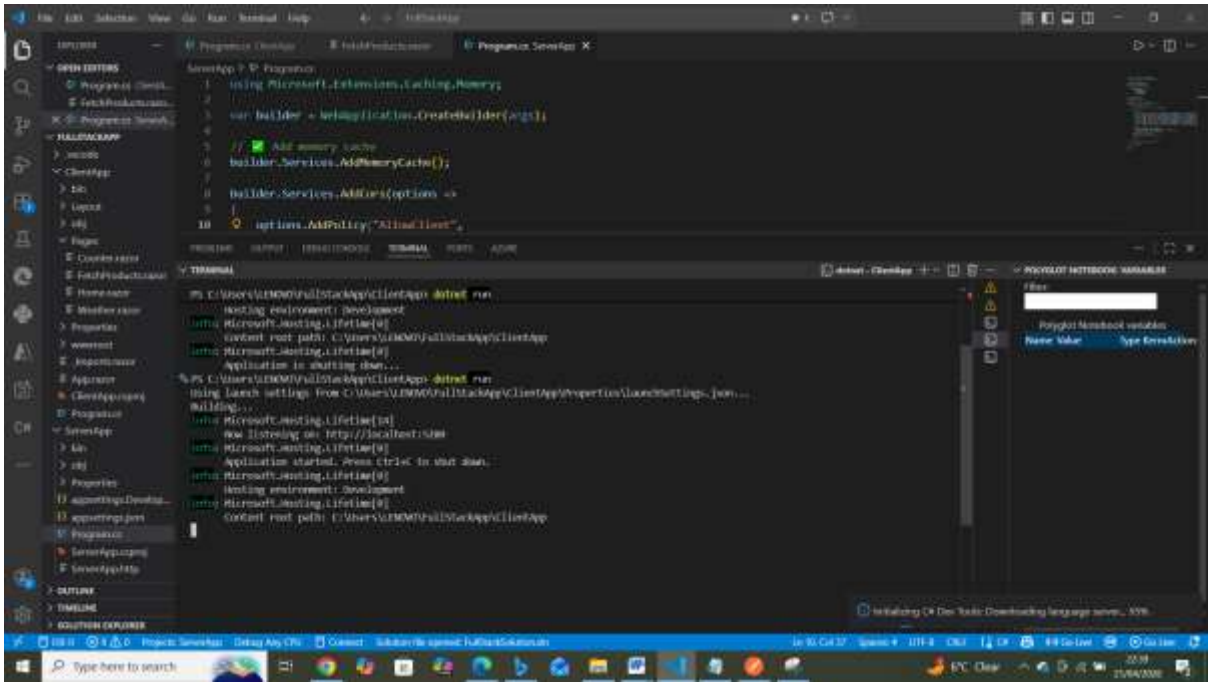
```
using Microsoft.Extensions.Caching.Memory;

var builder = WebApplication.CreateBuilder(args);

// 🧠 Add memory cache
builder.Services.AddMemoryCache();

builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowClient",
        policy => policy
            .AllowAnyOrigin()
            .AllowAnyMethod()
            .AllowAnyHeader());
});

var app = builder.Build();

app.UseCors("AllowClient");

// 🧠 Inject IMemoryCache into endpoint
app.MapGet("/api/products", (IMemoryCache cache) =>
{
    const string cacheKey = "products";

    if (!cache.TryGetValue(cacheKey, out object? cachedData))
    {
        var data = new[]
        {
```

Microsoft co-pilot AI helped me to cache the data.

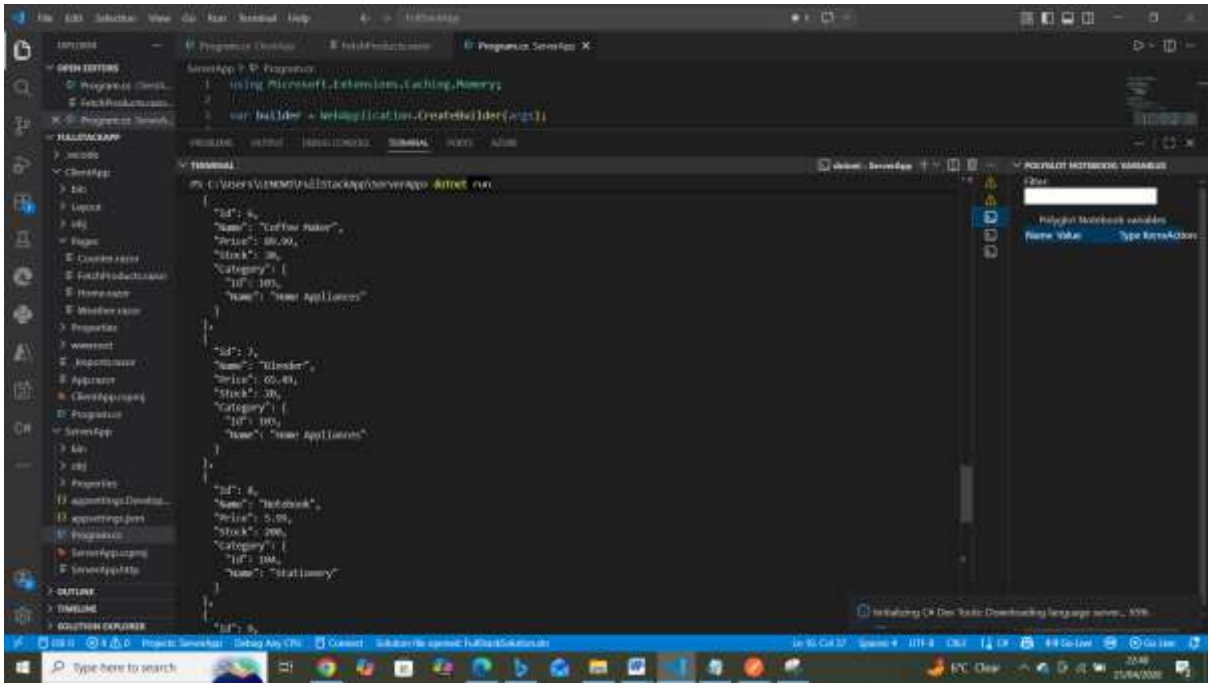
```
            new { Id = 1, Name = "Laptop", Price = 1200.50, Stock = 25,
Category = new { Id = 101, Name = "Electronics" } },
            new { Id = 2, Name = "Smartphone", Price = 799.99, Stock = 50,
Category = new { Id = 101, Name = "Electronics" } },
            new { Id = 3, Name = "Headphones", Price = 199.99, Stock =
100, Category = new { Id = 101, Name = "Electronics" } },
            new { Id = 4, Name = "Office Chair", Price = 150.00, Stock =
15, Category = new { Id = 102, Name = "Furniture" } },
            new { Id = 5, Name = "Desk Lamp", Price = 45.75, Stock = 40,
Category = new { Id = 102, Name = "Furniture" } },
            new { Id = 6, Name = "Coffee Maker", Price = 89.99, Stock =
30, Category = new { Id = 103, Name = "Home Appliances" } },
            new { Id = 7, Name = "Blender", Price = 65.49, Stock = 20,
Category = new { Id = 103, Name = "Home Appliances" } },
            new { Id = 8, Name = "Notebook", Price = 5.99, Stock = 200,
Category = new { Id = 104, Name = "Stationery" } },
            new { Id = 9, Name = "Pen Set", Price = 12.49, Stock = 150,
Category = new { Id = 104, Name = "Stationery" } },
            new { Id = 10, Name = "Backpack", Price = 39.99, Stock = 60,
Category = new { Id = 105, Name = "Accessories" } }
        };

        var options = new MemoryCacheEntryOptions()
            .SetAbsoluteExpiration(TimeSpan.FromMinutes(5))   // hard
expiry
            .SetSlidingExpiration(TimeSpan.FromMinutes(2));   // refresh
on access

        cache.Set(cacheKey, data, options);

        Console.WriteLine("⚡ Data loaded and cached");

        return data;
    }

    Console.WriteLine("⚡ Returned from cache");
```

*Microsoft co-pilot AI helped me to cache the data*

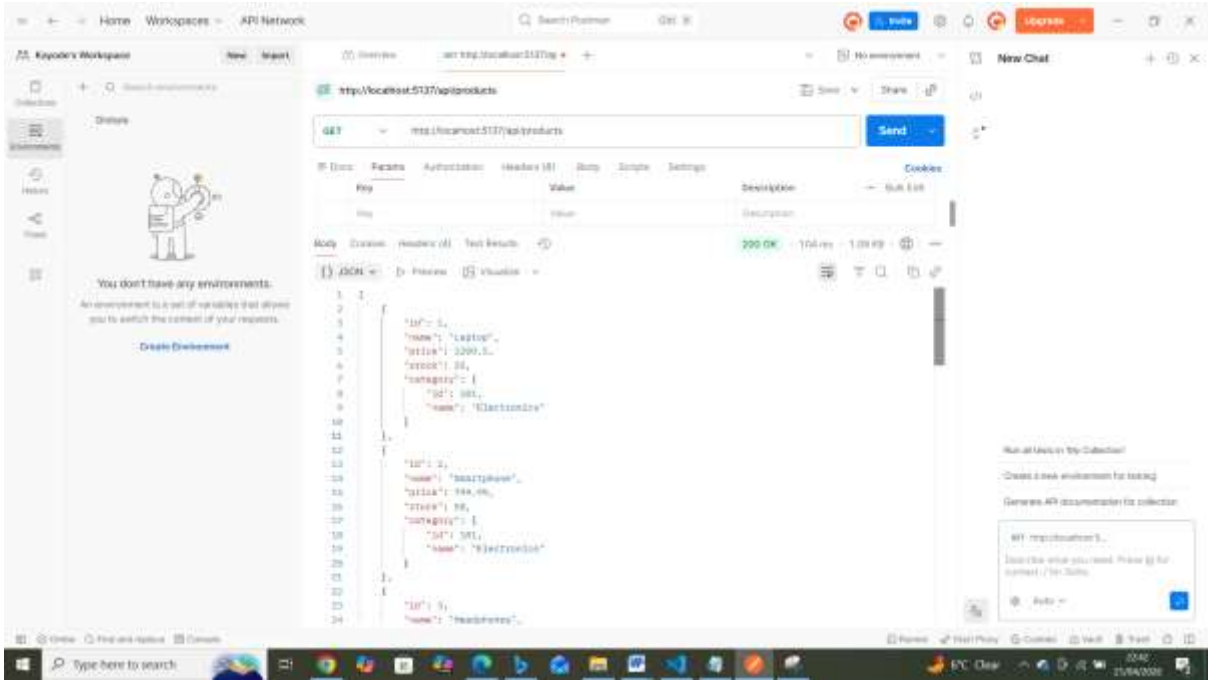
```
});

app.Run();
```

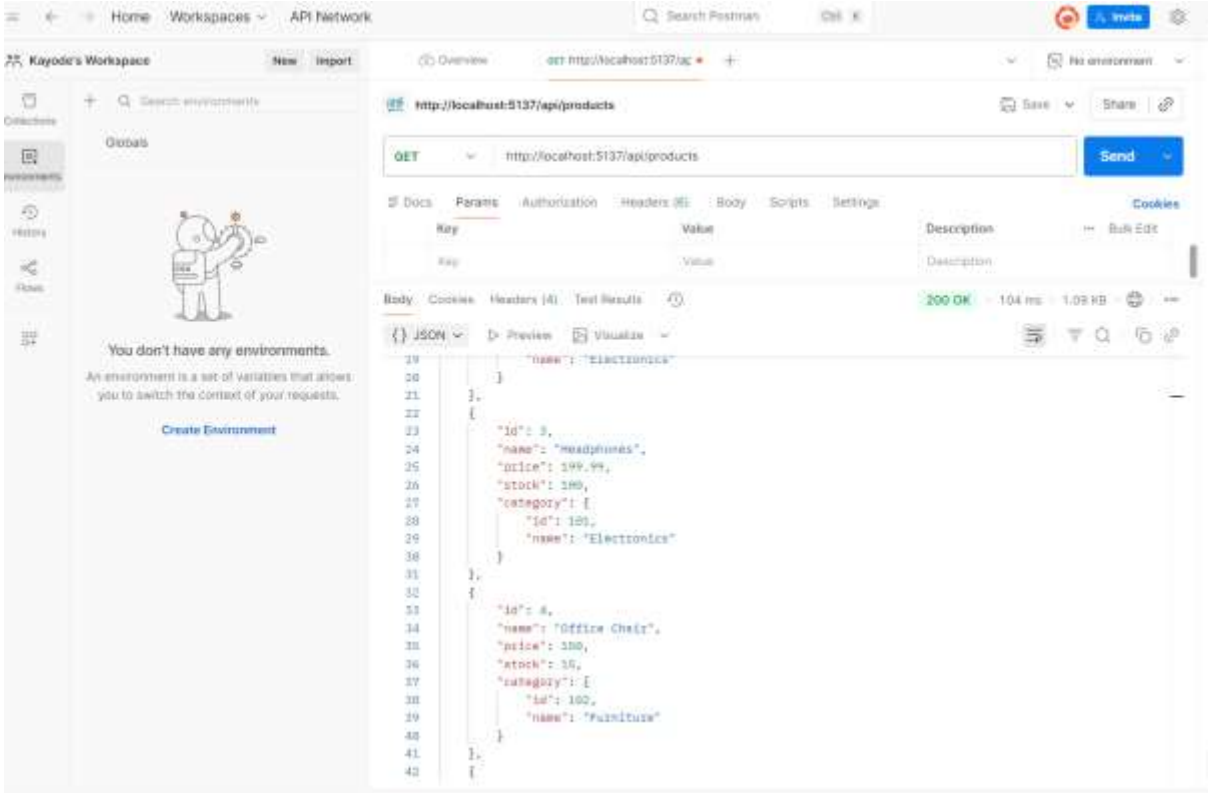


# SERVER SIDE CONSOLE DISPLAY

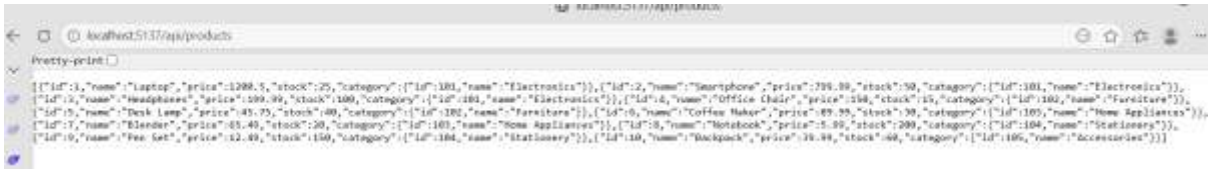


# USING POSTMAN




# Server side browser Json display



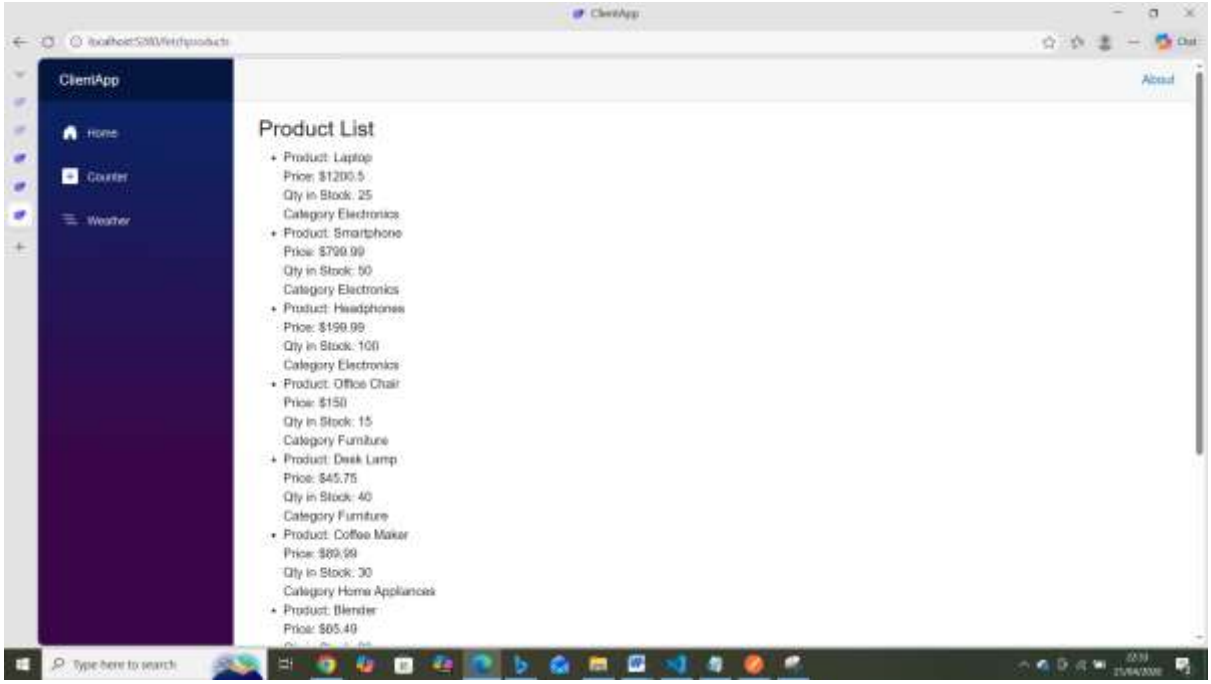
# The Pretty print looks like the following:

localhost:5137/api/products

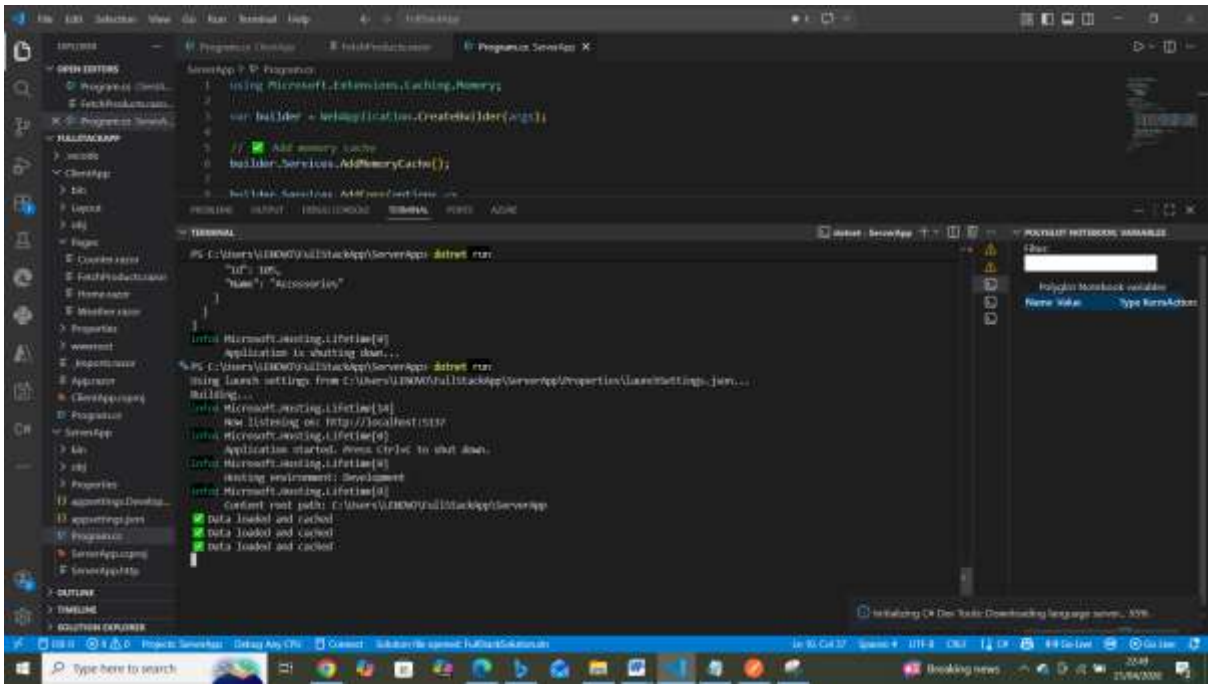
Pretty-print

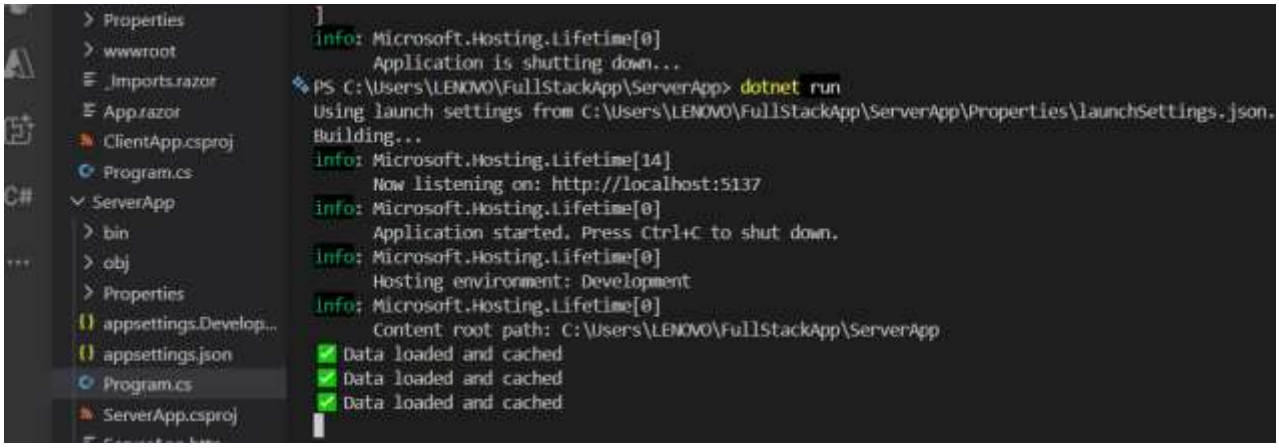

```
[
  {
    "id": 1,
    "name": "Laptop",
    "price": 1200.5,
    "stock": 25,
    "category": {
      "id": 101,
      "name": "Electronics"
    }
  },
  {
    "id": 2,
    "name": "Smartphone",
    "price": 799.99,
    "stock": 50,
    "category": {
      "id": 101,
      "name": "Electronics"
    }
  },
  {
    "id": 3,
    "name": "Headphones",
    "price": 199.99,
    "stock": 100,
    "category": {
      "id": 101,
      "name": "Electronics"
    }
  },
  {
    "id": 4,
    "name": "Office Chair",
    "price": 150,
    "stock": 15,
    "category": {
      "id": 102,
      "name": "Furniture"
    }
  },
  {
    "id": 5,
    "name": "Desk Lamp",
    "price": 45.75,
    "stock": 40,
    "category": {
```

# OUTPUT




# EVIDENCE OF CACHING THE DATA




Properties
wwwroot
_Imports.razor
App.razor
ClientApp.csproj
Program.cs
ServerApp
bin
obj
Properties
appsettings.Develop...
appsettings.json
Program.cs
ServerApp.csproj
]
info: Microsoft.Hosting.Lifetime[0]
Application is shutting down...
PS C:\Users\LENOVO\FullStackApp\ServerApp> dotnet run
Using launch settings from C:\Users\LENOVO\FullStackApp\ServerApp\Properties\launchSettings.json.
Building...
info: Microsoft.Hosting.Lifetime[14]
Now listening on: http://localhost:5137
info: Microsoft.Hosting.Lifetime[0]
Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
Content root path: C:\Users\LENOVO\FullStackApp\ServerApp
Data loaded and cached
Data loaded and cached
Data loaded and cached